

Prácticas de PSyD

Daniel Báscones García

19 de agosto de 2026

Índice

1. Configuración del entorno de programación	3
1.1. Paso 1: Configuración para máquina virtual	3
1.2. Paso 2: Creación de la Máquina Virtual	3
1.3. Paso 3: Configuración del ToolKit de STM	4
1.4. Paso 4: Descarga de manuales y recursos extra	4
2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)	6
2.1. Objetivos	6
2.2. Fundamentos teóricos	6
2.3. Desarrollo de la práctica	9
2.3.1. Creación del proyecto	9
2.3.2. Depuración del proyecto	12
2.3.3. Desarrollo del proyecto	15
2.4. Para hacer más	16
3. Práctica 2: Creación de un <i>Board Support Package</i> (BSP)	17
3.1. Objetivos	17
3.2. Fundamentos teóricos	17
3.2.1. Definiciones de direcciones	17
3.2.2. Definiciones de registros	18
3.2.3. Gestión de registros	19
3.2.4. Funciones de control de registros	19
3.2.5. Gestión directa bit a bit de registros	20
3.3. Desarrollo de la práctica	21
3.4. Para hacer más	22
4. Práctica 3: Entrada / Salida programada - UART TX	23
4.1. Objetivos	23
4.2. Fundamentos teóricos	23
4.2.1. UART	23
4.2.2. UART en el STM32F723IE	24
4.3. Desarrollo de la práctica	24
4.3.1. Envío de cadenas de texto por UART	26
4.4. Para hacer más	27
5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones	28
5.1. Objetivos	28
5.2. Fundamentos teóricos	28
5.2.1. Configuración de las interrupciones en el procesador	28

5.2.2. Gestión de interrupciones mediante funciones	29
5.3. Desarrollo de la práctica	29
5.4. Para hacer más	33
6. Práctica 5: Configuración de Timers	34
6.1. Objetivos	34
6.2. Fundamentos teóricos	34
6.3. Desarrollo de la práctica	35
6.4. Para hacer más	38
7. Práctica 6: Conexión con periféricos externos - Display y drivers	39
7.1. Objetivos	39
7.2. Fundamentos teóricos	39
7.2.1. Funcionamiento del periférico FMC	41
7.2.2. Funcionamiento del controlador ST7789H2	42
7.3. Desarrollo de la práctica	43
7.4. Para hacer más	45
8. Práctica 7: Buses de Expansión - I2C y pantalla táctil	46
8.1. Objetivos	46
8.2. Fundamentos teóricos	46
8.2.1. Funcionamiento del bus I2C - capa física	47
8.2.2. Direccionamiento en I2C - capa lógica	48
8.2.3. Líneas auxiliares a I2C	48
8.2.4. Conexiones en nuestra placa	49
8.3. Desarrollo de la práctica	49
8.3.1. Configuración del bus I2C	50
8.3.2. Interrupciones del CTP	51
8.3.3. Driver del I2C	52
8.3.4. Driver del CTP	54
8.3.5. Integración	56
8.4. Para hacer más	56
9. Práctica 8: <i>Bootloaders</i> y actualizaciones automáticas	57
9.1. Objetivos	57
9.2. Fundamentos teóricos	57
9.2.1. El módulo Wifi ESP-01S	59
9.3. Desarrollo de la práctica	59
9.3.1. Configuración previa al <i>bootloader</i>	59
9.3.2. Cesión del control a la aplicación	61
9.3.3. La memoria Flash	64
9.3.4. Programación del <i>bootloader</i>	65
9.3.5. Compilación de la aplicación	66
9.3.6. Configuración del servidor de descarga	66
9.4. Para hacer más	67

1. Configuración del entorno de programación

Las prácticas de la asignatura de PSyD se desarrollan en un entorno ya preparado dentro de una máquina virtual. Esto es debido a la comodidad y portabilidad que ofrece, ya que con trasladar un simple fichero, cualquiera puede tener todas las herramientas disponibles en su ordenador. No obstante, también se ofrece aquí una guía de instalación para quien quiera instalarlo todo de manera nativa en su ordenador. La guía se ha realizado para Ubuntu. Su adaptación a otros sistemas operativos (MacOS, o Windows) es relativamente directa, al estar las herramientas disponibles para todas las plataformas.

1.1. Paso 1: Configuración para máquina virtual

El software y los recursos necesarios ya están preinstalados en una máquina virtual basada en **Debian 13.6**. Cuenta con un entorno ligero (gestor de ventanas, navegador básico) y las **VirtualBox Guest Additions** para habilitar el passthrough de dispositivos USB (necesario para conectar la placa).

Puedes descargar la máquina virtual ya configurada y lista para importar en VirtualBox desde <https://drive.google.com/file/d/1U7Qvnkb-LfkJrcboyH4Jzs9iNczMntQg/view?usp=drivesdk>. También es necesario descargar **VirtualBox** para arrancarla. Para ello sigue las siguientes instrucciones:

- Asegúrate de tener instalado **VirtualBox** en su versión más reciente. Puedes descargarlo desde <https://www.virtualbox.org/wiki/Downloads>.
- Importante también descargar, desde el mismo enlace, el “VirtualBox Extension Pack”. Sin él, perderemos funcionalidades de la MV, entre otros, la capacidad de conectarle USBs (como el depurador ST-Link de la placa).

1.2. Paso 2: Creación de la Máquina Virtual

Sigue esta parte sólo si quieres crear una máquina virtual propia desde cero. Si quieres hacerlo de manera nativa, salta a Apartado 1.3.

Crea una nueva máquina virtual e instala **Debian** en su última versión (<https://www.debian.org/index.es.html>) o **Ubuntu** (recomendado versión **24.04 LTS**, descargable desde <https://ubuntu.com/download>). Configura el hardware asignándole los siguientes recursos recomendados:

- **Memoria RAM:** 4096 MB
- **Procesadores:** 2 núcleos (Cores).
- **Memoria de Vídeo:** 128 MB.
- **Tamaño del disco:** 60GB.
- **Tipo de instalación:** Manual. No utilizar la automática ya que instala cosas de más.

Una vez seleccionados los recursos, lanza la máquina y el instalador comenzará automáticamente. Deberás seleccionar cuestiones como el nombre de la máquina, usuario, contraseña... elige el que más te guste. El resto de la configuración se puede dejar por defecto. Ten en cuenta que el proceso de instalación puede tardar una una media hora (dependiendo del sistema anfitrión). Se recomienda utilizar las instalaciones gráficas para tener todo el sistema operativo bien preconfigurado. No obstante, también se pueden hacer instalaciones manuales para ajustar mejor las herramientas disponibles y, con ello, el tamaño de la máquina.

Una vez arranque la máquina, es importante instalar los extras de invitado (“Guest additions”), para conseguir que anfitrión y host se comunique adecuadamente (y puedan por ejemplo compartir portapapeles, ajustar el tamaño de pantalla, etc). Para ello haz click en **devices > insert guest additions CD image**. Esto cargará un disco en la MV (panel izquierdo). Abre una terminal en la ruta donde se monta el CD (normalmente /media/cdrom) y ejecuta:

```
1 # Actualizar paquetes existentes
2 > sudo apt update && sudo apt upgrade
3 # Instalar herramientas para compilar las guest additions
4 > sudo apt install build-essential dkms linux-headers-$
5 # Instalar guest additions
6 > sudo ./VBoxLinuxAdditions.run
```

1.3. Paso 3: Configuración del ToolKit de STM

Independientemente de si quieres utilizar tu propio sistema operativo para instalar las herramientas, o instalarlas en la máquina virtual, se hace de la misma manera. A continuación se muestran todas las herramientas, repositorios, etc necesarios para tener todo lo necesario en la asignatura. Si prefieres no realizar la instalación, descarga la máquina virtual ya configurada como se indica en el primer paso.

- Instalar STM32CubeIDE (la herramienta principal de desarrollo) desde aquí: <https://www.st.com/en/development-tools/stm32cubeide.html>
- Instalar STM32CubeMX (para configuración avanzada) desde aquí: <https://www.st.com/en/development-tools/stm32cubemx.html>
- Instalar STM32CubeProgrammer (para compatibilidad con zephyr) desde aquí: <https://www.st.com/en/development-tools/stm32cubeprog.html>
- Instalar picocom para comunicar con la placa con posterioridad:

```
1 > sudo apt-get install picocom
```

1.4. Paso 4: Descarga de manuales y recursos extra

Dentro de la MV ya creada, en la carpeta **Documentos** del entorno se encuentran varios recursos en forma de manuales. Si se está haciendo la instalación manual, estos son los recursos que ahí se encuentran y haría falta descargar:

- [rm0431](https://www.st.com/resource/en/reference_manual/rm0431-stm32f72xxx-and-stm32f73xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf) : https://www.st.com/resource/en/reference_manual/rm0431-stm32f72xxx-and-stm32f73xxx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf

Manual principal de referencia. Detalla toda la funcionalidad de los procesadores de la serie **STM32F723**, incluyendo periféricos, mapa de memoria, registros, modos de uso, etc.

- **um2140** : https://www.st.com/resource/en/user_manual/um2140-discovery-kit-with-stm32f723ie-mcu-stmicroelectronics.pdf
Manual con la descripción de conexiones y *pinout* detallado del kit de desarrollo **32F723EDISCOVERY**.
- **ds11853** : <https://www.st.com/resource/en/datasheet/stm32f722ic.pdf>
Funcionalidad genérica, especificaciones físicas y características electrónicas del procesador **STM32F723IE**.
- **pm0253** : https://www.st.com/resource/en/programming_manual/pm0253-stm32f7-series-and-stm32h7-series-cortexm7-processor-programming-manual-stmicroelectronics.pdf
Manual de programación de la serie **Cortex-M7**, con la referencia completa del conjunto de instrucciones ensamblador.
- **ST7789H2** : <https://www.crystallfontz.com/controllers/Sitronix/ST7789H2>
Manual de características del panel LCD ST7789H2 integrado en la placa.
- **an2606** : https://www.st.com/resource/en/application_note/cd00167594-stm32-microcontroller-system-memory-boot-mode-stmicroelectronics.pdf
Manual sobre las secuencias de inicialización (*bootloading*) en los dispositivos de ST.
- **DB3105**: https://www.st.com/resource/en/data_brief/32f723ediscovery.pdf
Resumen de características del kit de desarrollo **32F723EDISCOVERY**.

Adicionalmente, también se incluye en la MV creada la colección de ejemplos oficiales **STM32CubeF7**.

Nota: Esta colección utiliza librerías de alto nivel (HAL/LL). En la primera parte de las prácticas programaremos “desde cero” (a nivel de registro), por lo que no usaremos estas librerías directamente. Sin embargo, los ejemplos son un buen punto de referencia para explorar las capacidades de la placa.

- **STM32CubeF7 (GitHub)** <https://github.com/STMicroelectronics/STM32Cubef7>
Colección de ejemplos base. Incluye proyectos para múltiples placas, pero nos centraremos únicamente en la **32F723EDISCOVERY**.
Al clonar manualmente el repositorio, es imprescindible inicializar los submódulos para incluir todas las librerías requeridas. Mira bien las instrucciones del repositorio.
- **STM32CubeF7 Patch** <https://www.st.com/en/embedded-software/stm32cubef7.html>
Parche que se debe sobrescribir en el repositorio una vez clonado. Añade librerías propietarias de terceros necesarias para funciones como el control táctil o el procesamiento de audio.

2. Práctica 1: Periféricos mapeados en memoria (MMIO): Entrada/Salida de propósito general (GPIO)

2.1. Objetivos

- Comprender el concepto de **Memory-Mapped I/O (MMIO)**.
- Analizar la organización del mapa de memoria del microcontrolador **STM32F723IE**.
- Aprender a manipular directamente los registros de entrada/salida mediante máscaras de bits en C.

2.2. Fundamentos teóricos

En un procesador de propósito general, numerosas funciones se realizan a través de *periféricos*, componentes anexos al procesador con funciones específicas.

En la arquitectura ARM, los periféricos no se controlan con instrucciones dedicadas, sino que se comunican con la CPU a través del bus del sistema (*AXI-AHB Bus Matrix*) asignándoles direcciones físicas dentro del mapa de memoria de 32 bits (4GB).

Cada periférico posee un conjunto de **registros de control, estado y datos** ubicados en direcciones consecutivas a partir de una dirección base (*Base Address*). Escribiendo y leyendo en esas direcciones, conseguimos comunicarnos con los periféricos, que a su vez se conectarán con los diferentes pines del procesador. Los pines servirán para su conexión con el exterior con otros elementos, como LEDs, botones, pantallas, puertos, etc.

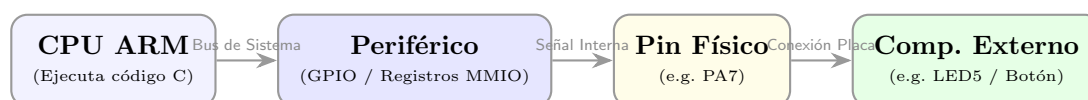


Figura 1: Flujo de control desde el código en CPU hasta el componente exterior a través de MMIO.

En nuestro caso, disponemos de un procesador **STM32F723IE**. Podemos consultar el manual de referencia [rm0431](#) para ver una lista completa de sus periféricos y las direcciones de memoria donde se ubican sus registros. La sección [rm0431 1.6](#) detalla todo este mapa de memoria, que se muestra en las Figuras 2 y 3

De aquí ya podemos deducir las direcciones de los diferentes controladores de GPIO. Vemos que están nombrados con letras (desde GPIOA hasta GPIOI). Por ejemplo, el controlador de GPIOA está en la dirección `0x40020000`. ¿Pero claro, cómo lo manejamos? Para ello debemos conocer todos los registros que se encuentran en esa zona de memoria. Esa información la tenemos en [rm0431 6.4](#).

A modo de ejemplo, se muestra en la Figura 4 la información referente al registro **MODER** del controlador de GPIO. Observamos que, en una palabra de 32 bits, contiene la configuración de modo de pin (Input/Output/Función alternativa/Analógico) para hasta 16 pines diferentes. Es decir, utiliza 2 bits para la configuración de cada pin. Los dos bits menos significativos corresponden al pin 0, y los 2 más significativos al pin 15, con el resto entre medias. Escribiendo en este registro configuraremos el modo de cada uno de estos pines.

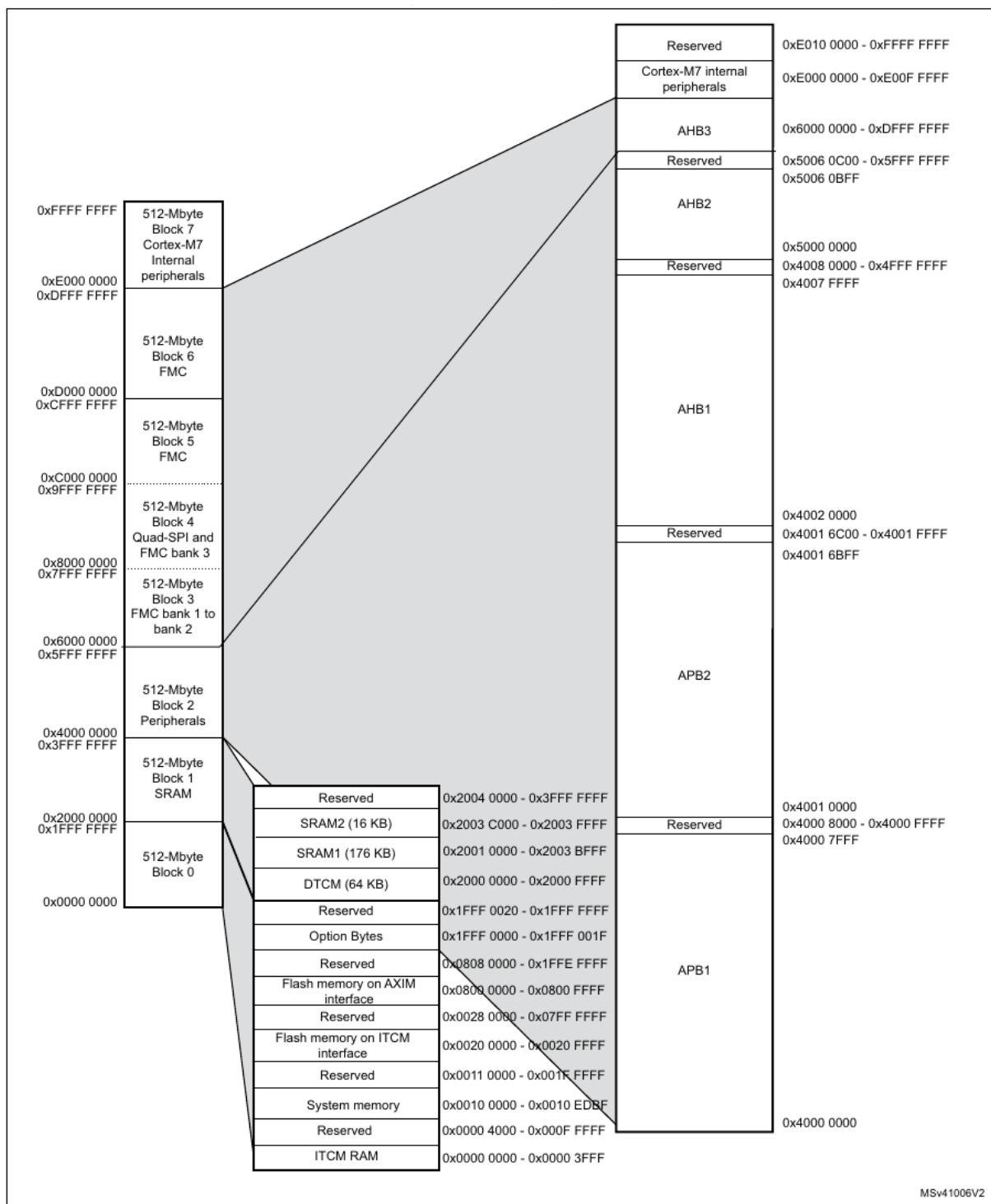


Figura 2: Mapa de memoria del **STM32F723IE**

Table 1. STM32F72xxx and STM32F73xxx register boundary addresses

Boundary address	Peripheral	Bus	Register map
0xA000 1000 - 0xA0001FFF	QuadSPI Control Register	AHB3	Section 13.5.14: QUADSPI register map on page 378
0xA000 0000 - 0xA000 0FFF	FMC control register		Section 12.8.6: FMC register map on page 349
0x5006 0800 - 0x5006 0BFF	RNG	AHB2	Section 16.7.4: RNG register map on page 458
0x5006 0000 - 0x5006 03FF	AES		Section 17.7.18: AES register map on page 510
0x5000 0000 - 0x5003 FFFF	USB OTG FS		Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4004 0000 - 0x4007 FFFF	USB OTG HS	AHB1	Section 32.15.64: OTG_FS/OTG_HS register map on page 1288
0x4002 6400 - 0x4002 67FF	DMA2		Section 8.5.11: DMA register map on page 252
0x4002 6000 - 0x4002 63FF	DMA1		
0x4002 4000 - 0x4002 4FFF	BKPSRAM		Section 5.3.27: RCC register map on page 192
0x4002 3C00 - 0x4002 3FFF	Flash interface register		Section 3.7.9: Flash interface register map
0x4002 3800 - 0x4002 3BFF	RCC		Section 5.3.27: RCC register map on page 192
0x4002 3000 - 0x4002 33FF	CRC		Section 11.4.6: CRC register map on page 275
0x4002 2000 - 0x4002 23FF	GPIOI		Section 6.4.11: GPIO register map on page 212
0x4002 1C00 - 0x4002 1FFF	GPIOH		
0x4002 1800 - 0x4002 1BFF	GPIOG		
0x4002 1400 - 0x4002 17FF	GPIOF		
0x4002 1000 - 0x4002 13FF	GPIOE		
0x4002 0C00 - 0x4002 0FFF	GPIOD		
0x4002 0800 - 0x4002 0BFF	GPIOC		
0x4002 0400 - 0x4002 07FF	GPIOB		
0x4002 0000 - 0x4002 03FF	GPIOA		

Figura 3: Detalle del mapa de memoria del **STM32F723IE**

6.4.1 GPIO port mode register (GPIOx_MODER) (x = A to K)

Address offset: 0x00

Reset value: 0xA800 0000 for port A

Reset value: 0x0000 0280 for port B

Reset value: 0x0000 0000 for other ports

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Bits 31:0 **MODER[15:0][1:0]**: Port x configuration I/O pin y (y = 15 to 0)

These bits are written by software to configure the I/O mode.

00: Input mode (reset state)

01: General purpose output mode

10: Alternate function mode

11: Analog mode

Figura 4: Detalle del registro MODER

Hemos visto pues que podemos configurar ciertos puertos de entrada/salida, que a su vez cuentan con numerosos pines. Pero, ¿en qué puerto/pin se sitúa, por ejemplo, un LED?. Para ello, debemos acudir a la documentación de la placa, donde se han diseñado físicamente las conexiones desde el procesador, hasta los diferentes elementos de soporte. En concreto, en la sección [um2140 5.16](#) tenemos la lista de todas las conexiones de botones y LEDs. Lo vemos en la Figura 5

Finalmente, sabemos que, por ejemplo, el LED5 se controla desde el puerto GPIOA, en el PIN 7. El puerto A está en la dirección `0x40020000`, y dispone de varios registros que deberemos configurar adecuadamente para controlar, finalmente, el LED.

2.3. Desarrollo de la práctica

2.3.1. Creación del proyecto

Vamos a controlar, directamente y escribiendo en registros del procesador, un LED (en concreto, el LED5 que acabamos de ver). Pero antes, debemos crear un proyecto en STMCubeIDE, la herramienta de desarrollo de ST. Para ello:

1. Abrimos STM32CubeIDE, buscándolo en el menú del sistema. Elegimos un workspace (o carpeta de trabajo) donde trabajar. Tras abrirlo, veremos el programa de la Figura 6
2. Creamos un nuevo proyecto con **File >STM32 Project Create / Import**.
3. Seleccionamos **STM32CubeIDE Empty Project** (Figura 7)
4. En **Board Selector**, buscar 723e y seleccionar la única placa disponible (Figura 8).

Table 5. Control port assignment

Reference	Color	Name	Comment
B1	BLUE	USER	Alternate function Wake-up
B2	BLACK	RESET	-
LD1	BLUE	ARDUINO	PA5
LD2	RED	5 V Power	-
LD3	RED	Fault Power	Current upper than 625 mA
LD4	RED/GREEN	ST-LINK COM	Green during communication
LD5	RED	USER1	PA7
LD6	GREEN	USER2	PB1
LD7	RED	USB OTG HS OVCR	PH10
LD8	GREEN	V _{BUS} USB HS	PB13
LD9	RED	USB OTG FS OVCR	PB10
LD10	GREEN	V _{BUS} USB FS	PA9

Figura 5: Pines para LEDS y botones de la placa **32F723EDISCOVERY**

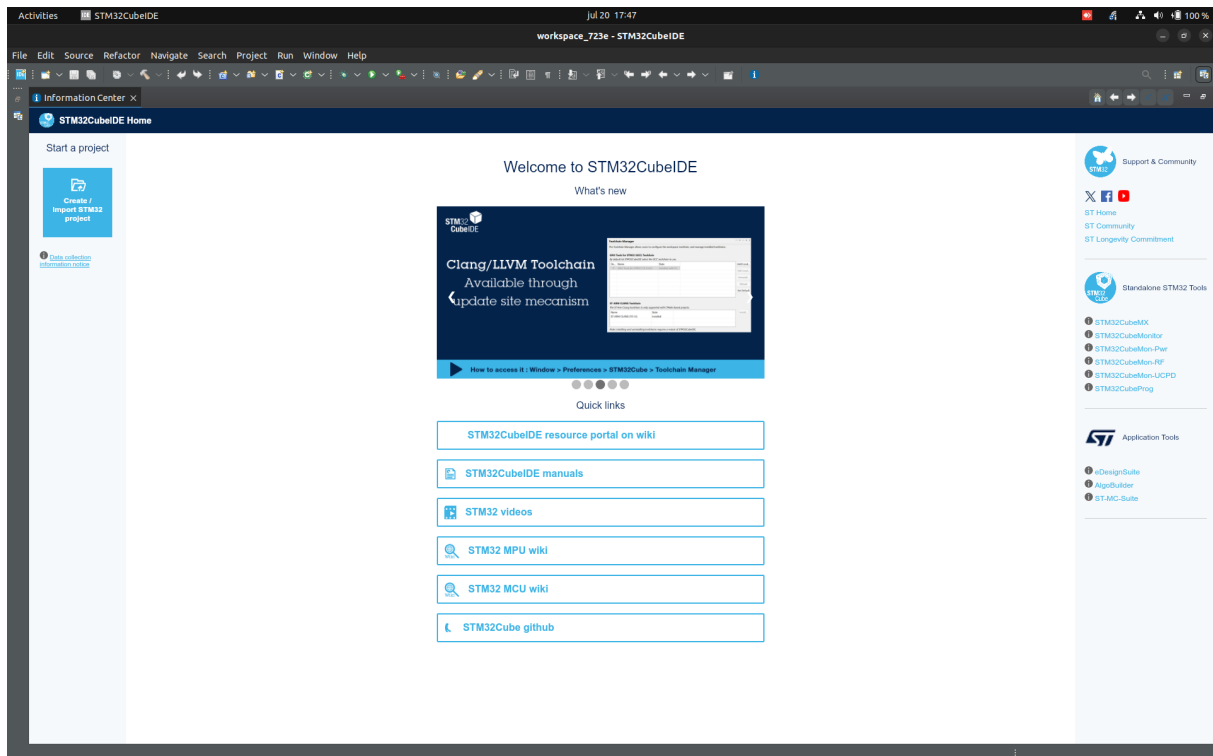


Figura 6: Ventana inicial de la aplicación

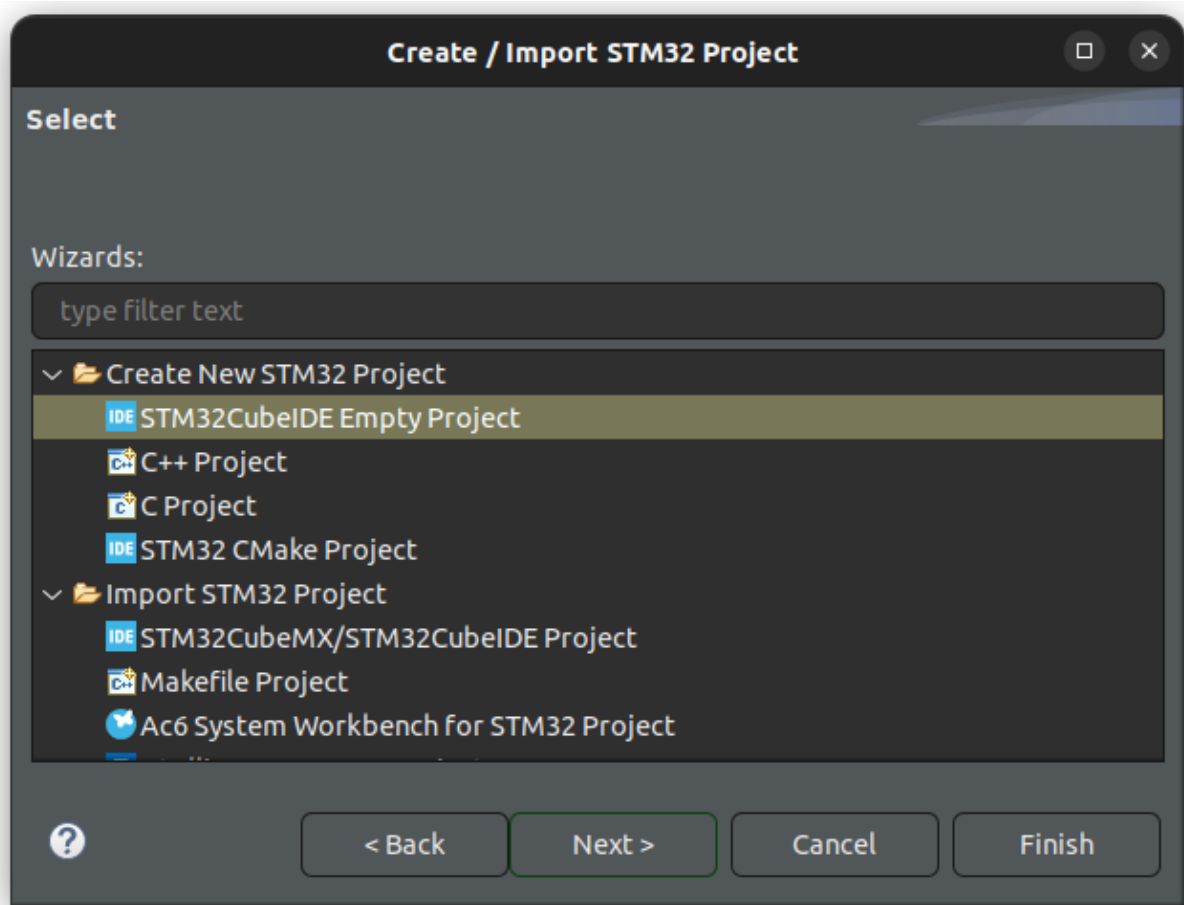


Figura 7: Creación de proyecto

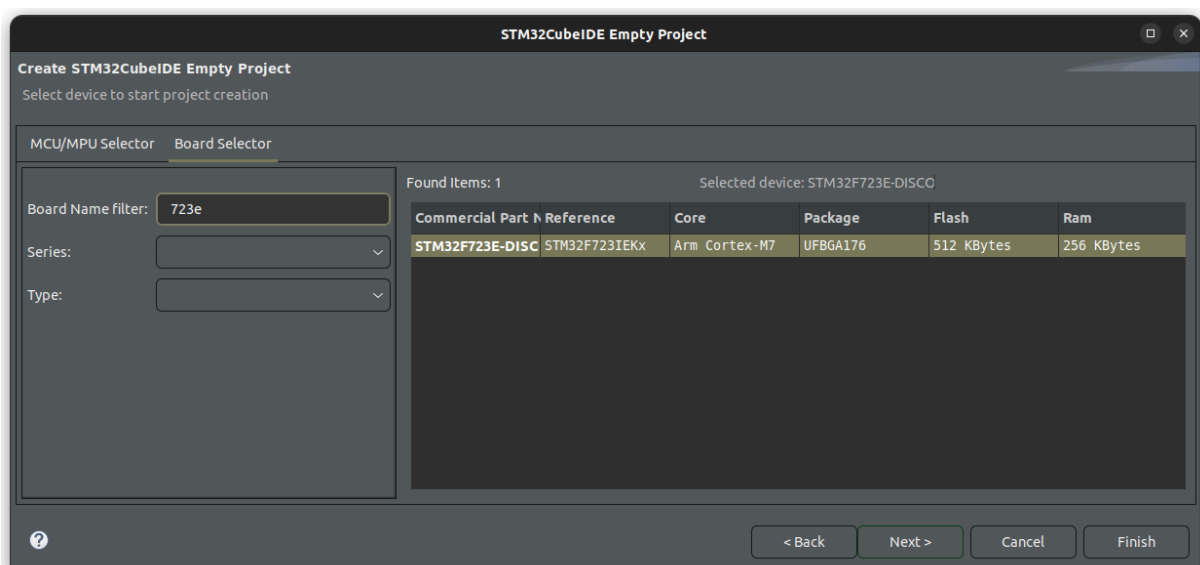


Figura 8: Selección de placa

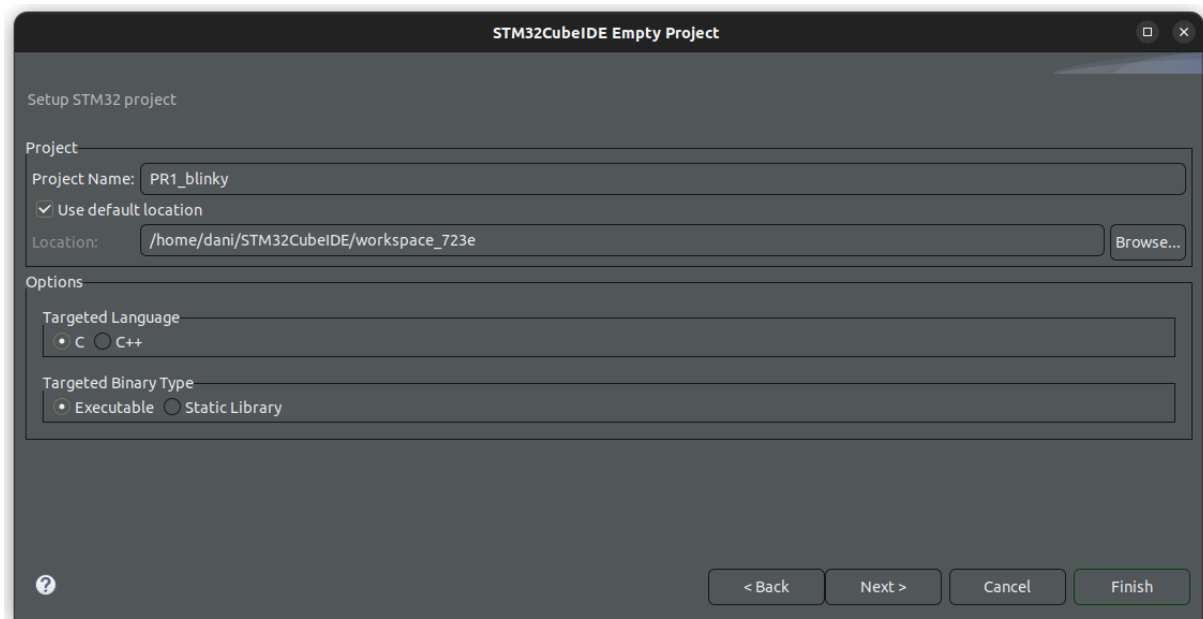


Figura 9: Selección de nombre de proyecto

5. Finalmente, dar un nombre al proyecto y terminar (Figura 9).

2.3.2. Depuración del proyecto

Ahora deberíamos tener un proyecto en blanco para trabajar (Figura 10). Antes de nada, vamos a probar que la conexión con la placa funciona, y podemos compilar y depurar el proyecto. Para ello:

1. Haz doble click en la carpeta **Src** dentro de tu proyecto, y luego, en **main.c**.
2. Con el archivo abierto, haz click en el martillo (build) que se encuentra en la barra superior de tareas. Verás el log de compilación en la terminal (Figura 11).
3. Finalmente, y si no ha habido problemas, haz click en el icono de depuración (debug) en la misma barra superior (el bichito verde).
4. Acepta la configuración de depuración por defecto (Figura 12) y vuelve a aceptar cuando pida cambiar a la vista de depuración.

Ahora puedes depurar el proyecto poco a poco. Entre otras, el depurador permite:

- Ir paso a paso por las líneas de código.
- Colocar *breakpoints* para parar la ejecución al llegar a ciertas líneas.
- Inspeccionar el contenido de los registros del procesador.
- Inspeccionar el contenido de los registros de los diferentes periféricos.
- Inspeccionar el contenido de la memoria.

Se recomienda explorar todas estas funcionalidades, que se pueden ver en la Figura 13.

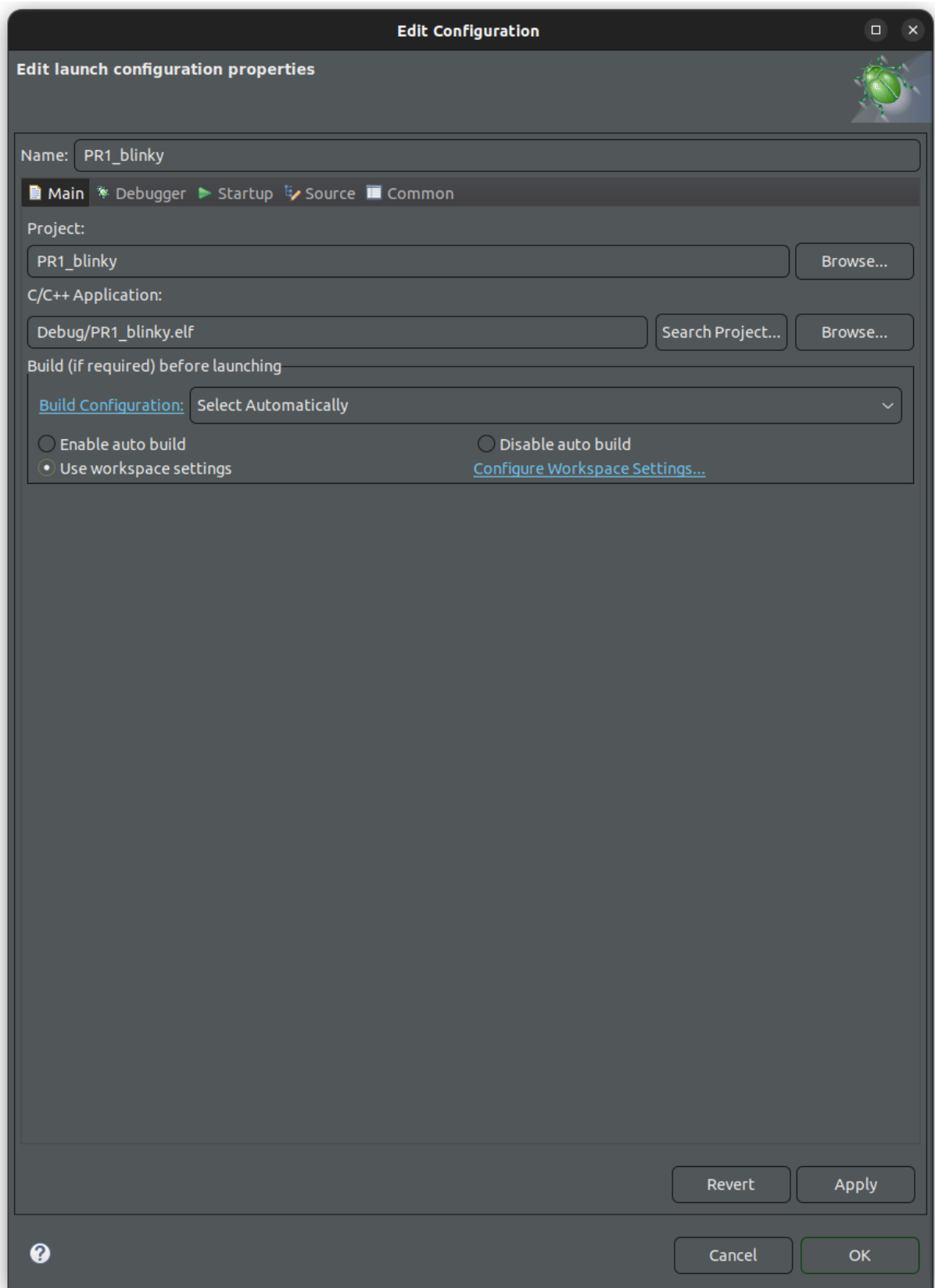


Figura 12: Selección de depuración por defecto

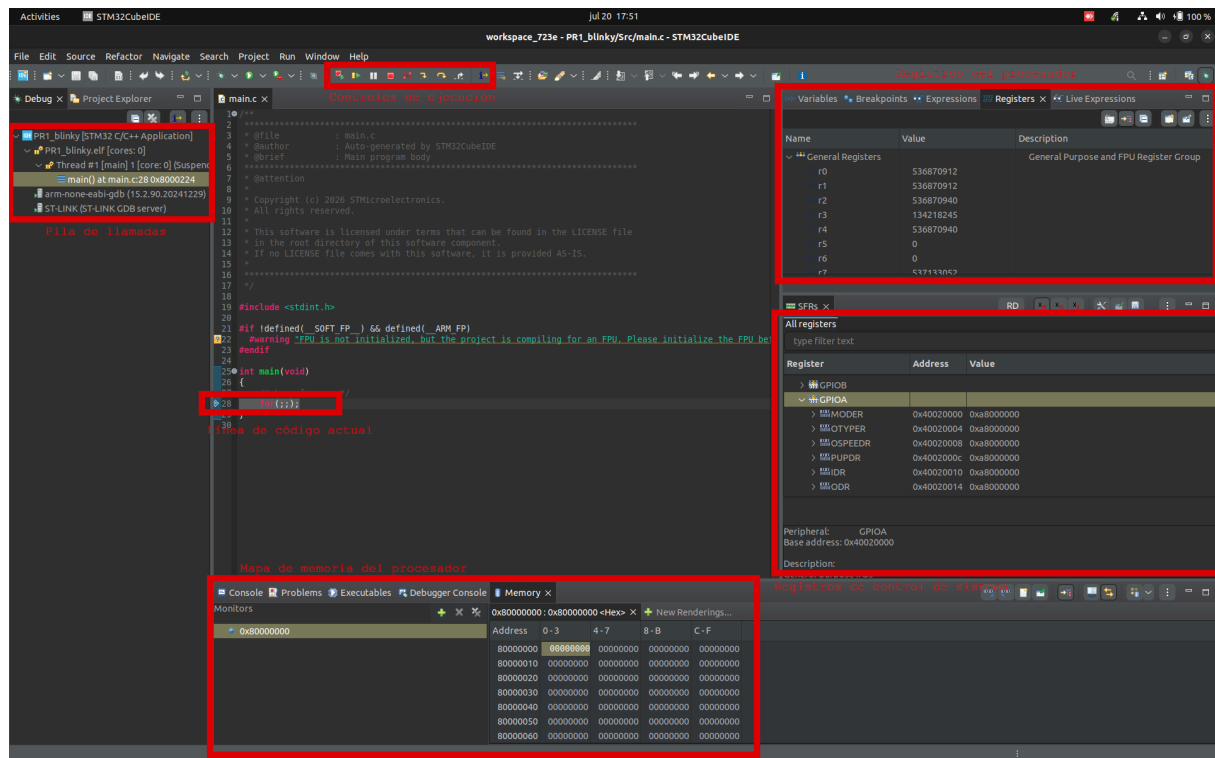


Figura 13: Ventana de depuración personalizada

2.3.3. Desarrollo del proyecto

Finalmente, debemos programar nuestro código para encender el LED. Para ello, habrá que escribir en las direcciones concretas de los registros de GPIO, en este caso de GPIOA-PIN7. Recuerda que la documentación de los registros disponibles está en [rm0431 6.4](#).

- Configurar GPIOA-PIN7 como salida en **MODER**.
- Poner el pin en tipo *push-pull* en **OTYPER**.
- Configurar la velocidad a alta en **OSPEEDR**.
- Deshabilitar las resistencias en **PUPDR**.
- Finalmente, se podrá escribir el valor del led (0/1) a través de **ODR**. Otra opción es escribir directamente al registro **BSRR**, que tiene la ventaja de evitar lecturas o escrituras múltiples (aunque para nuestro caso no afecta en nada).

Adicionalmente, tenemos que activar el reloj del periférico GPIOA a través del registro **RCC_AHB1ENR** ([rm0431 5.3.10](#)). La dirección base la podemos consultar en [rm0431 1.6.2](#).

Quizá te quede la pregunta de cómo narices escribo yo en una dirección de memoria concreta... la respuesta son... ¡punteros!

```
1 //Imaginemos que quiero escribir en la dirección 0x40020000
2 //justo en los bits 14-15
3 // (por lo que sea)
4
5 //borro configuración actual (máscara con ceros en las posiciones de interés)
```

```

6 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
7 //pongo la configuración que me interesa
8 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);

```

Aprovecha también esta función para hacer que parpadee cada cierto tiempo:

```

1 void delay(int loops) {
2     for (int i = 0; i < loops; i++) {
3         __asm("nop");
4     }
5 }

```

Nuestra función main.c tendrá por tanto esta pinta:

```

1 int main(void)
2 {
3     //activar reloj de GPIOA en RCC_AHB1ENR
4     //configurar Pin0-GPIOA como salida
5     //configurar Pin0-GPIOA modo push-pull
6     //configurar Pin0-GPIOA velocidad alta
7     //configurar Pin0-GPIOA con pullup/down desactivado
8
9     /* Loop forever */
10    for(;;) {
11        //encender LED
12        delay(1000000);
13        //apagar LED
14        delay(1000000);
15    }
16 }

```

2.4. Para hacer más

Busca el GPIO del botón de la placa, configúralo en modo entrada, y haz que el LED se encienda o apague cada vez que se pulse el botón, en lugar de hacerlo de manera automática.

3. Práctica 2: Creación de un *Board Support Package* (BSP)

3.1. Objetivos

- Desarrollar un *Board Support Package* minimalista.
- Entender la importancia de crear, en la medida de lo posible, código no dependiente del hardware.

3.2. Fundamentos teóricos

Un *Board Support Package* o BSP comprende una colección de bibliotecas que ayudan al programador a trabajar con una placa o procesador determinado. Generalmente, incluyen definiciones, funciones, macros y demás utilidades para que el programador no tenga que trabajar directamente con código de muy bajo nivel como, por ejemplo, hemos hecho en la práctica anterior. Por ejemplo:

```
1 // En lugar de un código así
2 * ((uint32_t *) (0x40020000)) &= ~(0b11 << (7*2));
3 * ((uint32_t *) (0x40020000)) |= 0b01 << (7*2);
4
5 // Buscamos algo así
6 GPIO_SET_MODE(GPIOA, PIN7, MODE_OUTPUT);
```

Por supuesto, alguien tiene que ser el encargado de crear, por primera vez, este BSP. En sistemas empuotrados, normalmente el BSP tiene que ser creado a medida, pues la combinación de procesador, periféricos y pines es particular para cada placa desarrollada. Para la placa **32F723EDISCOVERY**, ya existe un BSP, y se anima al alumno a consultarlo y ver cómo está construido. A lo largo de estas prácticas, desarrollaremos otro, mucho más minimalista y comedido, que aporte la funcionalidad justa que necesitamos. En esta práctica comenzamos con ese trabajo.

Vamos a ver el ejemplo concreto de cómo desarrollar la función anterior, para entender mejor las partes que puede tener un BSP.

3.2.1. Definiciones de direcciones

En primer lugar, necesitamos definir las direcciones de memoria donde se encuentran todos nuestros periféricos. Esto cumple una doble función: Por un lado, si cambiaran, las podemos modificar fácilmente al estar definidas como macros. Por otro lado, hacemos el código reutilizable para procesadores de la misma familia (por ejemplo, todos los Cortex-M7 de ARM (como el **STM32F723IE** que utilizamos) reutilizan partes de los mismos BSPs).

```
1 #define GPIO_BASE_ADDR 0x40020000
2 #define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
3 #define GPIOB_BASE_ADDR (GPIO_BASE_ADDR + 0x0400)
4 ...
```

Table 24. GPIO register map and reset values

Offset	Register name	31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0x00	GPIOA_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	1	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x00	GPIOB_MODER	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0
0x00	GPIOx_MODER (where x = C..K)	MODER15[1:0]		MODER14[1:0]		MODER13[1:0]		MODER12[1:0]		MODER11[1:0]		MODER10[1:0]		MODER9[1:0]		MODER8[1:0]		MODER7[1:0]		MODER6[1:0]		MODER5[1:0]		MODER4[1:0]		MODER3[1:0]		MODER2[1:0]		MODER1[1:0]		MODER0[1:0]	
	Reset value	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
0x04	GPIOx_OTYPER (where x = A to K)	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
	Reset value																	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

Figura 14: Extracto parcial de registros del controlador de GPIO

3.2.2. Definiciones de registros

Lo siguiente es definir los registros que se encuentran en dichas direcciones. Como suelen ser varios, y consecutivos, se suele definir una estructura de tipo (en lenguaje C) que los englobe todos, por facilidad de uso. Por ejemplo, mirando [rm0431 6.4.11](#), podemos ver todos los registros contenidos en cada controlador de GPIO. Se muestra la primera parte de la tabla de registros en la Figura 14

Un detalle interesante a ver aquí es que, por ejemplo, para el registro `MODER`, nos dice que tiene tres valores por defecto (reset) diferentes, según si se trata del puerto A, B, u otros. El registro `OTYPER`, sin embargo, tiene el mismo valor por defecto para todos los puertos (A-K). En cada registro, además, nos indica qué bit controla qué cosa. Para los registros `MODER`, los bits, de dos en dos, controlan el modo de GPIO (especificados en [rm0431 6.4.1](#)). Para `OTYPER`, controla el tipo de salida ([rm0431 6.4.2](#)). Si hacemos el código para tener una estructura que nos permita controlar estos registros, tendríamos algo de este estilo:

```

1 typedef struct
2 {
3     volatile uint32_t MODER;    /*!< GPIO port mode register, Address offset: 0
4     volatile uint32_t OTYPER;  /*!< GPIO port output type register, Address
5     ...                          offset: 0x04 */
6 } GPIO_TypeDef;

```

Importante observar la declaración de las variables como tipo `volatile`, ya que, además de modificarse internamente desde el procesador, son registros cuyo valor puede modificarse desde fuera (por el propio control del periférico). Con la palabra clave `volatile` conseguimos que el compilador no optimice esas variables y siempre las consulte directamente de memoria.

Pregunta: ¿A qué se debe que todos los registros queden colocados en direcciones sucesivas de memoria al ser declarados con esta sintaxis?

Pregunta: Explora qué hace exactamente el compilador al trabajar con objetos volátiles.

3.2.3. Gestión de registros

Tenemos definidas las direcciones, y las estructuras de registros. Ahora necesitamos tener una forma de escribir en dichas estructuras. Para ello, vamos a crear punteros a las direcciones de memoria base, con el tipo de estructura definido:

```
1 #define GPIOA ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
```

Si la base del controlador se encuentra en la dirección `0x40020000` (como es el caso de GPIOA), los registros se encontrarán a continuación (el primero `MODER` en `0x40020000`, el segundo `OTYPER` en `0x40020004`, y así sucesivamente). Así, lo que conseguimos es que, posteriormente, en el código de nuestro programa, podamos hacer algo de este estilo:

```
1 GPIOA->MODER = 0xA8000000;  
2 uint32_t temp = GPIOA->MODER;
```

Exactamente equivalente a haber escrito (o leído) desde las direcciones de memoria correspondientes. Es decir, de una forma legible, acceder (para escribir o leer) los contenidos de estos registros, que a la vez sirven para control de los periféricos.

Pregunta: Cada registro normalmente controla, desde cada uno de sus bits, elementos diferentes del periférico. ¿Cómo puedo leer (o escribir) en unos bits concretos del registro?

3.2.4. Funciones de control de registros

Aunque ahora es más cómodo trabajar con los registros, lo ideal es contar con funciones que nos ayuden, aún más, a abstraer todos estos códigos de bajo nivel que son propensos a errores por parte del programador. Por ejemplo, ahora mismo, si quisiéramos cambiar el modo del pin 7, del puerto A, a *output*, deberíamos hacer algo de este estilo:

```
1 GPIOA->MODER &= ~(0b11 << (7*2));  
2 GPIOA->MODER |= 0b01 << (7*2);
```

Es decir, borramos primero los dos bits de las posiciones 14 y 15 (Ver tabla anterior) y luego les ponemos el valor "01" para configurarlo a nuestro gusto. Pero, ¿y si ahora queremos cambiar el pin 4 del puerto B? ¿o ponerlo en modo *input*?. Es mejor crear una función genérica que nos permita hacer todo.

Aunque esto ya es a gusto del consumidor, la sugerencia para estas funciones es hacerlas lo más genéricas posibles. En este caso, deberíamos poder permitir trabajar con cualquier puerto, cualquier pin, y cualquier modo. Por ejemplo, podemos hacer algo así:

5.3.10 RCC AHB1 peripheral clock register (RCC_AHB1ENR)

Address offset: 0x30

Reset value: 0x0010 0000

Access: no wait state, word, half-word and byte access.

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	OTGHS ULPIEN	OTGHS EN	Res.	Res.	Res.	Res.	Res.	Res.	DMA2 EN	DMA1 EN	DTCMRA MEN	Res.	BKPSR AMEN	Res.	Res.
	rw	rw							rw	rw	rw		rw		
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	Res.	CRC EN	Res.	Res.	Res.	GPIOI EN	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GIOD EN	GPIOC EN	GPIO BEN	GPIO AEN
			rw				rw	rw	rw	rw	rw	rw	rw	rw	rw

Figura 15: Detalle de bits del registro **AHB1ENR**

```
1 typedef enum {
2     GPIO_MODE_INPUT    = 0x00,
3     GPIO_MODE_OUTPUT   = 0x01,
4     GPIO_MODE_ALT      = 0x02,
5     GPIO_MODE_ANALOG   = 0x03
6 } GPIO_Mode_t;
7
8 static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin,
9     GPIO_Mode_t mode) {
10     uint32_t temp = GPIOx->MODER; //leemos registro
11     temp &= ~(0b11 << (pin * 2)); //borramos modo actual
12     temp |= (mode << (pin * 2)); //ponemos modo nuevo
13     GPIOx->MODER = temp; //escribimos de vuelta
14 }
```

Con esto, ya podríamos hacer desde nuestro código principal, lo siguiente:

```
1 GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
```

3.2.5. Gestión directa bit a bit de registros

Otros registros, como por ejemplo **AHB1ENR** ([rm0431 5.3.10](#)) tienen una estructura que no es regular, ya que cada bit controla una cosa diferente, como podemos ver en la Figura 15.

Para estos casos, C nos permite una sintaxis como la siguiente, que nos deja acceder individualmente a nivel de bit a los registros:

```
1 //Base address
2 #define RCC_AHB1EN_BASE_ADDR 0x40023830
3
4 //Structure for Reset and Clock Control peripheral clock register 1
5 typedef union {
6     volatile uint32_t reg;
7
8     // 2. Access individual bits or bit-groups for the register above
9     struct {
10         volatile uint32_t GPIOAEN : 1; // Bit 0: Enable peripheral (1 bit
11     )
12 }
```

```

11         volatile uint32_t GPIOBEN      : 1;  // Bit 1: Enable peripheral (1 bit
12         )
13     } bits;
14 } RCC_AHB_ENR_TypeDef;
15
16 //Handle:
17 #define RCC_AHB1ENR      ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)

```

Posteriormente, podemos acceder a los campos individuales de la siguiente manera:

```

1 RCC_AHB1ENR->bits.GPIOAEN = 1;

```

Este registro en concreto es muy importante, pues activa los periféricos. De no activarlos, no funcionarán aunque se configuren adecuadamente.

3.3. Desarrollo de la práctica

Crea un BSP para que, en el código de la práctica anterior, no haya que acceder directamente a registros del periférico GPIOA. Necesitarás funciones para configurar **MODER**, **OTYPER**, **OSPEEDR**, **PUPDR** y también para escribir 1 o 0 (esto lo puedes hacer desde el registro **ODR** o **BSRR**). Consulta [rm0431 6.4](#) para ver todos los registros y sus funcionalidades. Deberás:

1. Completar la definición de tipo para incluir todos los registros. También completar la definición de tipo de **AHB1ENR**.

```

1 //... completar direcciones
2 #define GPIO_BASE_ADDR
3 #define GPIOA_BASE_ADDR (GPIO_BASE_ADDR + 0x0000)
4 #define RCC_AHB1EN_BASE_ADDR
5
6 //Structure for GPIO controllers
7 typedef struct
8 {
9     volatile uint32_t MODER;    /*!< GPIO port mode register,
10                                Address offset: 0x00    */
11     volatile uint32_t OTYPER;   /*!< GPIO port output type register,
12                                Address offset: 0x04    */
13     //... completar
14 } GPIO_TypeDef;
15
16 //Structure for Reset and Clock Control peripheral clock register 1
17 typedef union {
18     volatile uint32_t reg;
19
20     // 2. Access individual bits or bit-groups for the register above
21     struct {
22         volatile uint32_t GPIOAEN      : 1;  // Bit 0: Enable peripheral
23         (1 bit)
24         volatile uint32_t GPIOBEN      : 1;  // Bit 1: Enable peripheral
25         (1 bit)
26         //... completar
27     } bits;
28 } RCC_AHB_ENR_TypeDef;
29
30 #define GPIOA      ((GPIO_TypeDef *) GPIOA_BASE_ADDR)
31 #define RCC_AHB1ENR ((RCC_AHB_ENR_TypeDef *) RCC_AHB1EN_BASE_ADDR)

```

2. Crear las funciones, junto con sus definiciones de valores, para controlar los diferentes registros. Toma el ejemplo previo de la función `GPIO_MODE_SET` para hacer las demás.

```
1 static inline void GPIO_MODE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
    , GPIO_Mode_t mode);
2 static inline void GPIO_OTYPE_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_OType_t otype);
3 static inline void GPIO_OSPEED_SET(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_OSpeed_t ospeed);
4 static inline void GPIO_PUPD_SET(volatile GPIO_TypeDef *GPIOx, uint32_t pin
    , GPIO_PuPd_t pupd);
5 static inline void GPIO_PIN_WRITE(volatile GPIO_TypeDef *GPIOx, uint32_t
    pin, GPIO_State_t state);
```

3. Recuerda empaquetar toda esta funcionalidad en un fichero `bsp.h` que irá dentro de la carpeta `Inc` del proyecto.
4. Adaptar el código original `main.c` de la práctica para utilizar el BSP.

```
1 int main(void)
2 {
3     RCC_AHB1ENR->bits.GPIOAEN = 1;
4
5     GPIO_MODE_SET(GPIOA, 7, GPIO_MODE_OUTPUT);
6     //... completar
7
8     for(;;) {
9         GPIO_PIN_WRITE(GPIOA, 7, GPIO_STATE_ONE);
10        delay(1000000);
11        //... completar
12        delay(1000000);
13    }
14 }
```

3.4. Para hacer más

Abre el proyecto BSP de ejemplo (dentro del paquete `STM32CubeF7`) e investiga cómo están organizadas las funciones, definiciones, registros... Para navegar por el proyecto, puedes ir haciendo `ctrl+click` en las llamadas a funciones o variables, y te llevará al fichero donde están definidas.

4. Práctica 3: Entrada / Salida programada - UART TX

4.1. Objetivos

- Comprender más en profundidad el concepto de "periférico" dentro de un procesador.
- Aprender a programar un periférico.
- Interactuar con el periférico desde fuera de la placa.

4.2. Fundamentos teóricos

4.2.1. UART

La comunicación **UART** (*Universal Asynchronous Receiver-Transmitter*) es un protocolo de comunicación serie asíncrono utilizado ampliamente en sistemas embebidos. Al ser asíncrono, no requiere de una señal de reloj compartida entre el emisor y el receptor; en su lugar, ambos dispositivos deben estar configurados previamente a la misma velocidad de transmisión (*baud rate*).

Estructura de la Trama UART:

En estado de reposo (*Idle*), la línea de transmisión se mantiene en nivel lógico alto (**1**). Una trama estándar de datos está compuesta por los siguientes elementos:

1. **Bit de Inicio (*Start Bit*):** Transición de nivel alto (**1**) a nivel bajo (**0**) durante un período de bit. Notifica al receptor que una nueva trama está comenzando.
2. **Longitud de Palabra (*Data Bits*):** Bloque de datos útil, transmitido habitualmente empezando por el bit menos significativo (*LSB first*). Suele configurarse entre 5 y 9 bits, siendo 8 bits el valor más común.
3. **Bit de Paridad (*Parity Bit, opcional*):** Bit añadido al final de los datos para la detección elemental de errores:
 - **Paridad Par (*Even*):** Se ajusta a 1 o 0 para que el número total de unos en la palabra de datos (incluyendo el bit de paridad) sea par.
 - **Paridad Impar (*Odd*):** Se ajusta para que el número total de unos sea impar.
4. **Bits de Parada (*Stop Bits*):** Uno o varios bits en nivel lógico alto (**1**) al final de la trama que garantizan un tiempo de resincronización mínimo para la línea antes de la siguiente transmisión.

Diagrama de Tiempos Típico:

En la Figura 16 se muestra la representación gráfica de una trama serie de 8 bits de datos con paridad y 1 bit de stop:

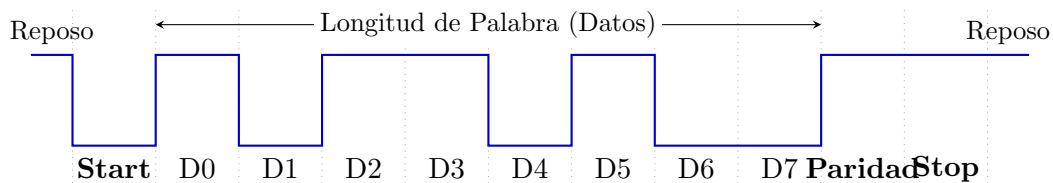


Figura 16: Cronograma de una trama de datos UART de 8 bits con paridad y 1 bit de parada.

4.2.2. UART en el STM32F723IE

En el procesador de la placa, se disponen de varios periféricos de UART diferentes, numerados como USARTx, siendo la x el número de UART. La UART dispone de varios registros de control, otros de configuración, y finalmente registros para la recepción/envío de datos. Todos ellos se describen en [rm0431 27.8](#). Las direcciones base de los periféricos USARTx se encuentran en [rm0431 1.6.2](#). Cómo hacer la transmisión en sí se describe en [rm0431 27.5.2](#).

¿Ahora, qué USARTx utilizamos? en [um2140 5.13](#) nos dice que la USART6 está conectada directamente a la salida desde ST-Link (El depurador que venimos utilizando hasta ahora para conectar con la placa). Por tanto, deberemos configurar USART6. Los pines donde se conecta (que habrá que activar) se pueden ver en [ds11853 Table 10](#). En concreto, vemos que son el pin PC6 para USART6.TX y PC7 para USART6.RX. (De momento solo usamos el TX para transmitir). Para que por ese pin salga la UART (Y no el valor genérico que ponemos en el puerto de salida del GPIO), deberemos configurar la función alternativa 8 como indica [ds11853 Table 12](#), que conecta la USART6 al puerto C.

4.3. Desarrollo de la práctica

Para el desarrollo de esta práctica hay que extender el BSP introduciendo las funciones y definiciones necesarias de la UART, y posteriormente controlarla para emitir datos hacia el ordenador. La documentación necesaria puede verse en [rm0431 27](#). Los pasos a seguir son los siguientes:

1. Según [ds11853 Table 10](#), el pin PC6 es quien saca al exterior la UART. Configúralo en modo de función alternativa en [MODER](#), tipo push-pull en [OTYPER](#), pull-up en [PUPDR](#), velocidad alta en [OSPEEDR](#), y configura la función alternativa en [AFR](#) para que sea la 8 (como indica [ds11853 Table 12](#)). No te olvides de activar el puerto C a través de [AHB1ENR](#).

```
1 RCC_AHB1ENR->bits.GPIOCEN = 1;
2 GPIO_MODE_SET(GPIOC, 6, GPIO_MODE_ALT);
3 //... completar
```

2. Ahora le toca el turno a la USART6 como nos indica [um2140 5.13](#). Actívala desde [APB2ENR](#) lo primero.

```
1 //en bsp.h
2 #define RCC_APB2EN_BASE_ADDR 0x40023844
3 //Structure for Reset Control peripheral register APB1
4 typedef union {
5     volatile uint32_t reg;
6
7     // 2. Access individual bits or bit-groups for the register above
```



```

8     struct {
9         volatile uint32_t TIM1EN      : 1;  // Bit 0:
10        volatile uint32_t TIM8EN      : 1;  // Bit 1:
11        volatile uint32_t RES1        : 2;  // Bit 2-3:
12        //... completar
13    } bits;
14 } RCC_APB2R_TypeDef;
15 #define RCC_APB2ENR                ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)
16
17
18 //en main.c
19 RCC_APB2ENR->bits.USART6EN = 1;          //enable UART clock

```

3. Configura la UART desde los registros **CR1** y **CR2**. Activa el periférico, configura la longitud de palabra a 8 bits, 1 bit de stop y sin paridad, pone por último el oversampling a 16.
4. Configura la velocidad de la UART desde **BRR**. Tendrás que ponerle el valor del reloj del sistema (16MHz) entre la velocidad de la uart (115200).
5. Por último, activa el modo de transmisión desde **CR1**.

```

1  // UART STOP BITS
2  typedef enum {
3      UART_STOP_ONEBIT = 0x00,
4      UART_STOP_HALFBIT = 0x01,
5      UART_STOP_TWOBIT  = 0x02,
6      UART_STOP_ONEHALFBIT = 0x03
7  } UART_Stop_t;
8  #define UART_STOP_MASK 0b11
9  #define UART_STOP_SHIFT 12
10
11 static inline void UART_STOPBIT_SET(volatile USART_TypeDef *USARTx,
12     UART_Stop_t stopbits) {
13     //... completar configurar el registro CR2 con los bits de stop
14 }
15
16 // UART ENABLE
17 typedef enum {
18     UART_DISABLE = 0x00,
19     UART_ENABLE = 0x01
20 } UART_Enable_t;
21 #define UART_ENABLE_MASK 0b1
22 #define UART_ENABLE_SHIFT 0
23
24 static inline void UART_ENABLE_SET(volatile USART_TypeDef *USARTx,
25     UART_Enable_t enable) {
26     //... completar configurar el registro CR1 con el bit de enable
27 }
28
29 // UART WORD LENGTH
30 typedef enum {
31     UART_WORD_8B = 0x00,
32     UART_WORD_9B = 0x01,
33     UART_WORD_7B = 0x02
34 } UART_WordLength_t;
35 #define UART_WORD_MASK 0b1
36 #define UART_WORD_SHIFT 12
37
38 static inline void UART_WORDLENGTH_SET(volatile USART_TypeDef *USARTx,
39     UART_WordLength_t length) {
40     //... completar escribir longitud en registro CR1

```

```

38 }
39
40 //... completar todas estas otras funciones auxiliares
41 static inline void UART_PARITY_SET(volatile USART_TypeDef *USARTx,
    UART_Parity_t parity);
42 static inline void UART_OVERSAMPLING_SET(volatile USART_TypeDef *USARTx,
    UART_Oversampling_t over);
43 static inline void UART_TX_MODE_SET(volatile USART_TypeDef *USARTx,
    UART_TX_Mode_t mode);

```

6. Cuando quieras transmitir un byte, espera primero a que quede libre el transmisor (aparece un 1 en el bit TC del registro **ISR**). A continuación escribe el dato en el registro **TDR**, la UART lo transmitirá solo.

```

1 //... en bsp.h
2 static inline void UART_TransmitByte(volatile USART_TypeDef *USARTx,
    uint8_t data)
3 {
4     while (!(USARTx->ISR & (1U << 7U))); //Wait until we can push data (TXE
        bit enabled)
5     USARTx->TDR = data; //push data (this clears TXE bit)
6 }
7
8
9 //... en main.c
10 UART_TransmitByte(USART6, 'X');
11 UART_TransmitByte(USART6, 'Y');
12 //...

```

Para ver ahora los mensajes desde el ordenador, utilizaremos la utilidad **picocom** (o cualquier otra terminal de vuestro gusto). Desde la máquina virtual, abrir una terminal y ejecutar el comando:

```

1 picocom /dev/ttyACM0 -b 115200 -d 8 -y n -p 1

```

NOTA: El nombre del puerto puede no ser necesariamente **ttyACM0**, aunque en linux al menos es el valor por defecto. En Windows, por ejemplo, suele llamarse **COMx**, siendo x un número arbitrario.

4.3.1. Envío de cadenas de texto por UART

Aunque mola mucho programar en *bare-metal*, también es útil partir de librerías ya implementadas para facilitar la vida a los programadores. Podemos ampliar la funcionalidad de nuestra UART para imprimir mensajes de depuración (así en próximas prácticas podremos depurar vía mensajes, en ocasiones más cómodo e intuitivo que la depuración con *breakpoints*). Incluye las siguientes funciones y prueba a depurar tu código con mensajes.

En **bsp.h** tendrás que añadir una función para escribir cadenas completas:

```

1 void UART_WriteString(USART_TypeDef *usart, const char *str) {
2     while (*str) {
3         UART_TransmitByte(usart, *str++);
4     }
5 }

```

En `main.c` tendrás que añadir la función de depuración en sí con las librerías necesarias auxiliares:

```
1 #include <stdint.h>
2 #include <stdlib.h>
3 #include <stdio.h>
4 #include <string.h>
5 #include <stdarg.h>
6
7 void Debug_Printf(const char *fmt, ...)
8 {
9     char buf[256];
10    va_list args;
11    va_start(args, fmt);
12    vsnprintf(buf, sizeof(buf), fmt, args);
13    va_end(args);
14
15    UART_WriteString(USART6, buf);
16 }
```

Y listo! Puedes llamar a la función `Debug_Printf` con el texto con formato que quieras, para imprimir los valores de variables, por ejemplo, o simplemente colocar mensajes de depuración por la UART.

4.4. Para hacer más

Investiga las interrupciones de la UART para evitar tener que hacer esperas activas que consuman recursos de CPU.

5. Práctica 4: Entrada / Salida por interrupciones - UART RX y botones

5.1. Objetivos

- Aprender qué son las interrupciones, cómo funcionan, y sus características.
- Programar el controlador de interrupciones para la recepción de datos con UART.
- Programar el controlador de interrupciones para entrada con botón.

5.2. Fundamentos teóricos

5.2.1. Configuración de las interrupciones en el procesador

Para responder de manera rápida y consistente a eventos en un microcontrolador, es buena idea utilizar una *interrupción hardware* para ejecutar un pequeño fragmento de código al detectar dichos eventos. En esta práctica, vamos a ver cómo funciona el controlador de interrupciones en STM32, que puede configurarse para ejecutar ciertas funciones al ocurrir determinados eventos.

Cada procesador tiene su propia manera de tratar interrupciones. En la familia STM32, y en concreto de **STM32F723IE**, se utilizan dos periféricos diferentes para su gestión:

- **NVIC** : El (*Nested Vectored Interrupt Controller*) [pm0253 4.2](#) es el gestor de interrupciones interno del procesador Cortex-M7, núcleo dentro del **STM32F723IE**. Permite la programación de hasta 240 interrupciones, con control de prioridad en cada una de ellas. En el caso de nuestro procesador, **STM32F723IE**, están disponibles unas 100 ([rm0431 9.1.2](#)), aunque utilizaremos nada más que un par de ellas.
- **EXTI** : (*EXtended Interrupt Controller*). Su propósito es extender el número de interrupciones disponibles a los programadores a través de eventos externos al propio procesador. Cada periférico interno tiene sus propias líneas de interrupción en el **NVIC** , como puede verse en [rm0431 9.1.2](#) , sin embargo, para los eventos en pines de E/S (a través de GPIO), la configuración de la interrupción se hace desde el **EXTI** .

En la familia STM32, los pines GPIO están agrupados (PA0, PB0, PC0...). El periférico **EXTI** no tiene una línea directa para cada pin individual de la placa, sino que usa líneas compartidas (la línea **EXTI0** gestiona PA0, PB0, PC0, etc. a través de un multiplexor en el **SYSCFG**). Por tanto, no se pueden usar, por ejemplo, PA0 y PB0 como interrupciones independientes simultáneamente.

Adicionalmente, los periféricos externos, aunque estén conectados a las líneas de interrupción, deben ser configurados para generar las señales de interrupción necesarias.

En resumen, estos procesadores tienen interrupciones internas a través del **NVIC** , que hay que activar, además de realizar la configuración de los periféricos y, algunos casos como el GPIO, configurar **EXTI** para recibir sus interrupciones. En esta práctica, configuraremos las interrupciones de **UART** directamente desde **NVIC** , y las de GPIO utilizando también **EXTI** . Podemos ver un diagrama de las interconexiones entre todo en la Figura 17.

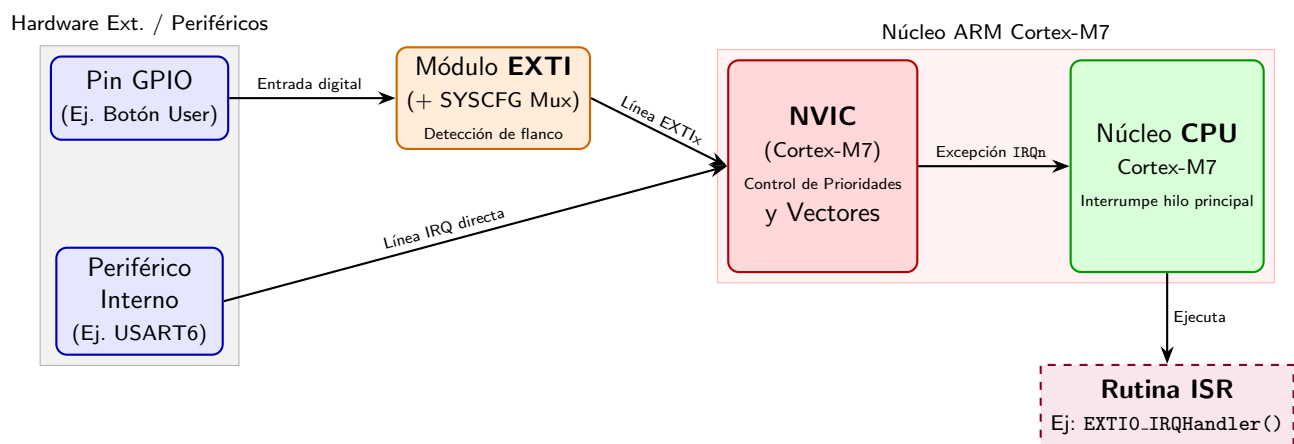


Figura 17: Flujo de interrupciones en el microcontrolador STM32F723E.

5.2.2. Gestión de interrupciones mediante funciones

Cuando se produce una interrupción, el procesador debe pausar su ejecución y saltar a la dirección de memoria donde se encuentra la rutina de respuesta o ISR (Interrupt Service Routine). Como la ubicación física de estas funciones en la memoria Flash no es fija (depende de la compilación), el procesador utiliza una Tabla de Vectores de Interrupción (Vector Table).

Esta tabla es un array de punteros ubicado al inicio de la memoria (por defecto desde `0x00000000`), donde cada posición está reservada para un evento concreto:

- La posición `0x00000000` almacena el valor inicial del puntero de pila (MSP).
- La dirección `0x00000004` contiene el vector de Reset.
- Direcciones posteriores contienen los vectores o punteros a las rutinas de interrupción, como por ejemplo `0x0000015C` que contiene el puntero a la rutina de la `USART6` ([rm0431 9.1.2](#)).

Para evitarnos la tarea de configurar esta tabla manualmente, el código de inicio del procesador (`Startup/startup_stm32f723ieqx.s`) la genera automáticamente durante el arranque. Por defecto, todas las entradas de la tabla apuntan a un bucle infinito genérico (`Default_Handler`) mediante el atributo `.weak`. Si definimos en nuestro código C una función con el nombre exacto de la interrupción (por ejemplo, `USART6_IRQHandler`), el enlazador (linker) la sobrescribirá automáticamente, redirigiendo la tabla hacia nuestra rutina. Podemos ver este proceso en Figura 18.

5.3. Desarrollo de la práctica

Como en prácticas anteriores, el objetivo es seguir extendiendo nuestro BSP con las funciones necesarias como para controlar los periféricos y funcionalidades que utilizaremos en esta práctica. En este caso, debemos hacer varias cosas:

1. Apúntate las direcciones base de los periféricos `NVIC` en [pm0253 4.2](#), y `EXTI` y `SYSCFG` en [rm0431 1.6.2](#), crea además las estructuras de datos con las que accederás a los registros. Básate, respectivamente, en [pm0253 4.2](#) para `NVIC` (Ten en cuenta que los

1. Tabla de Vectores (Flash / RAM)

Offset / Dir.	Contenido (Vector)
0x00000000	Stack Pointer
0x00000004	Reset_Handler
...	...
0x00000058	EXTIO_IRQHandler
...	...
0x0000015C	USART6_IRQHandler
...	...

2. Funciones en Flash (.text)

Dirección	Código / Función
0x08000100	Reset_Handler()
0x08000250	Default_Handler()
...	...
0x08001F10	USART6_IRQHandler()
0x08002F4C	EXTIO_IRQHandler()

Atributo .weak en Startup:
Si el usuario **NO** define la función en C, la tabla apunta al Default_Handler (bucle infinito).

Redefinición en C:
Al escribir void USART6_IRQHandler(void) en C, el linker asigna la dirección real de tu función en la tabla.

Figura 18: Mapeo de la Tabla de Vectores de Interrupción a la memoria del programa.

registros no están consecutivos), en [rm0431 10.7.7](#) para **EXTI**, y en [rm0431 7.2.8](#) para **SYSCFG**. Crea además las variables para manejarlos. Puedes basarte en el siguiente código:

```

1 #define EXTI_BASE_ADDR      0x40013C00UL
2 //... completar para NVIC y SYSCFG
3
4 typedef struct
5 {
6     volatile uint32_t ISER[8U];           /*!< Offset: 0x000 (R/W)
7         Interrupt Set Enable Register */
8     uint32_t RESERVED0[24U];
9     //... completar
10 } NVIC_TypeDef;
11
12 typedef struct
13 {
14     volatile uint32_t MEMRMP;             /*!< SYSCFG memory remap register,
15                                         Address offset: 0x00 */
16     //... completar
17 } SYSCFG_TypeDef;
18
19 typedef struct
20 {
21     volatile uint32_t IMR;                /*!< EXTI Interrupt mask register,
22                                         Address offset: 0x00 */
23     //... completar
24 } EXTI_TypeDef;
25
26 #define EXTI                ((EXTI_TypeDef *) EXTI_BASE_ADDR)
27 //... completar para NVIC y SYSCFG

```

- Debemos activar el puerto y pin correspondiente al botón. Revisando la documentación, en [um2140 Table 5](#) nos dice que el botón está enchufado a la función wake-up del procesador, y en [um2140 Table 19](#) que dicha función SYS_WKUP1 está en el PA0. Por tanto, lo primero es configurar este pin como entrada, de tipo *push-pull* con el *pulldown* activado, y a velocidad alta. No te olvides de activar también el reloj de GPIOA a través de [AHB1ENR](#).

3. A continuación, hay que activar también el pin de transmisión de la UART. En [um2140 Table 19](#) podemos observar que la función `USART6_RX` está conectada al pin `PC7`. Tenemos que configurarlo en modo *alternativo* con la función 8 según nos indica [ds11853 Table 12](#). Adicionalmente, configurarlo en modo *push-pull* con resistencia de *pullup* y velocidad alta.
4. Finalmente, configuramos la `UART` para que también funcione en modo recepción y tenga activas las interrupciones de recepción. Ambas cosas se pueden hacer desde el registro `CR1` ([rm0431 27.8.1](#)). Crea funciones en el BSP y define las constantes necesarias. Puedes hacer algo de este estilo:

```

1 //en main.c
2 UART_RX_MODE_SET(USART6, UART_RX_ENABLE);
3 UART_RX_INT_SET(USART6, UART_RX_INT_ENABLE);
4
5
6 //en bsp.h
7 // RECEIVE enable
8 typedef enum {
9     UART_RX_DISABLE = 0x00,
10    UART_RX_ENABLE  = 0x01
11 } UART_RX_Mode_t;
12 #define UART_RX_MODE_MASK 0b1
13 #define UART_RX_MODE_SHIFT 2
14
15 static inline void UART_RX_MODE_SET(volatile USART_TypeDef *USARTx,
16    UART_RX_Mode_t mode) {
17     //... completar tocar el bit adecuado de CR1
18 }
19
20 //... completar hacer una función similar para RX INT enable

```

5. Ahora ya podemos comenzar a activar las interrupciones. Lo primero es activar el reloj para la configuración del sistema en `APB2ENR` ([rm0431 5.3.14](#)), para lo cual previamente deberemos definir los campos del registro.

```

1 //en bsp.h
2 #define RCC_APB2EN_BASE_ADDR 0x40023844 //APB2 clock control register
3
4 //Structure for Reset Control peripheral register APB1
5 typedef union {
6     volatile uint32_t reg;
7
8     // 2. Access individual bits or bit-groups for the register above
9     struct {
10         volatile uint32_t TIM1EN      : 1; // Bit 0:
11         volatile uint32_t TIM8EN      : 1; // Bit 1:
12         volatile uint32_t RES1        : 2; // Bit 2-3:
13         //... completar
14     } bits;
15 } RCC_APB2R_TypeDef;
16
17 #define RCC_APB2ENR ((RCC_APB2R_TypeDef *) RCC_APB2EN_BASE_ADDR)

```

A continuación, debemos configurar la interrupción externa `EINT0` (que multiplexa todos los pines 0) para que trabaje con el puerto A (recordemos que el botón está en `PA0`). Esto se hace desde el registro `EXTICR` de `SYSCFG` ([rm0431 7.2.3](#)). Finalmente, hay que activar la interrupción externa 0 desde `EXTI` a través del registro `IMR` ([rm0431 10.9.1](#)) en modo *rising trigger* en el registro `RTSR` ([rm0431 10.9.3](#)).

```

1 //... completar en bsp.h crear y rellenar estas funciones

```

```

2 static inline void SYSCFG_EINT_MAP_PORT(uint32_t eintno, uint32_t portno) {
3     //... completar modificar SYSCFG->EXTICR
4 }
5
6 static inline void EXTI_ENABLE_RISING_TRIGGER(uint32_t eintno, uint32_t
7     enable) {
8     //... completar activar/desactivar EXTI->RTSR
9 }
10
11 static inline void EXTI_UNMASK_INTERRUPT(uint32_t eintno, uint32_t unmask)
12 {
13     //... completar activar/desactivar EXTI->IMR
14 }
15
16 //en main.c
17 RCC_APB2ENR->bits.SYSCFGEN = 1;
18 //Activar EINT0 PUERTO A
19 SYSCFG_EINT_MAP_PORT(0, 0); //port a (0) to eint0
20 EXTI_ENABLE_RISING_TRIGGER(0, 1); //enable on port a
21 EXTI_UNMASK_INTERRUPT(0, 1); //unmask on port a

```

6. Ya tendríamos con esto el pin PA0 generando una interrupción en EXTI0, pero el procesador aún no responde a estas interrupciones. Adicionalmente, necesitamos activar la interrupción de la **UART**. Ambas cosas se hacen desde el **NVIC** en el registro **ISER** ([pm0253 4.2.2](#)). Para saber qué bits activar, debemos consultar la numeración de las líneas de interrupción en [rm0431 9](#).

```

1 //en bsp.h
2 //... completar buscar los números de línea
3 #define EXTI0_LINE_IRQn
4 #define USART6_INT_IRQn
5
6 static inline void NVIC_ENABLE_INT(uint32_t line) {
7     //... completar modificar el registro NVIC->ISER
8 }
9
10 //en main.h
11 NVIC_ENABLE_INT(EXTI0_LINE_IRQn);
12 NVIC_ENABLE_INT(USART6_INT_IRQn);

```

7. Por último, necesitamos crear dos funciones que respondan a sendas interrupciones. En este caso, la función **USART6_IRQHandler** que responda a la **USART6**, y **EXTI0_IRQHandler** que responda al botón.
8. En el caso de la primera, **USART6_IRQHandler**, deberemos comprobar primero que la interrupción ha sido generada por recepción, consultando el bit **RXNE** del registro **ISR** de la **UART** ([rm0431 27.8.8](#)). Si es así, podremos leer el registro **RDR** ([rm0431 27.8.10](#)) para consultar el dato recibido, lo cual a su vez limpiará la interrupción. Dentro de la interrupción, para darle funcionalidad, podemos por ejemplo enviar de vuelta el dato leído al ordenador.

```

1 void USART6_IRQHandler(void)
2 {
3     //... completar comprobar si el flag de interrupción está activo
4     if () {
5         //... completar leer el byte
6     }
7 }

```


9. En el caso de la segunda `EXTIO_IRQHandler`, el proceso es diferente. Primero, debemos comprobar que la interrupción está pendiente desde el registro `PR` de `EXTI` ([rm0431 10.9.6](#)). A continuación, habrá que borrar dicho bit, escribiendo de nuevo en el registro, como indica la documentación. En esta función, por ejemplo, se puede poner como funcionalidad el dar la vuelta al LED que ya teníamos configurado anteriormente, para comprobar que funciona.

```
1 void EXTI0_IRQHandler(void)
2 {
3     //... comprobar en EXTI->PR si la línea 0 generó interrupción
4     if () {
5         //... limpiar línea 0 escribiendo la posición 0 de EXTI->PR
6         //... completar funcionalidad
7     }
8 }
```

5.4. Para hacer más

Intenta cambiar “en caliente” la función que gestiona una interrupción, por ejemplo la del botón, modificando el puntero de las direcciones de memoria a las que salta el `NVIC`.

6. Práctica 5: Configuración de Timers

En esta práctica vamos a introducir el concepto de temporizador. Son unos periféricos del procesador que permiten, cada cierto tiempo, generar interrupciones y, con ello, disparar funcionalidades concretas que hayamos programado. En este caso programaremos uno para hacer parpadear un LED, y otro diferente para utilizar como *debouncer* del botón.

6.1. Objetivos

- Entender el funcionamiento de los temporizadores internos del sistema.
- Interconectar interrupciones de varios periféricos para lograr funcionalidades avanzadas.

6.2. Fundamentos teóricos

Un temporizador o *timer* en un microcontrolador es un periférico hardware que, en esencia, es un contador digital. La cuenta se realiza en función de una señal de reloj de entrada, y al llegar a ciertos valores (configurables) dispara interrupciones.

Es capaz de medir intervalos de tiempo de forma independiente a la ejecución de código en la CPU, liberándola pues para que haga otras tareas en lugar de esperas activas a ciertos eventos. Los temporizadores suelen contar con varios registros para su control, entre otros:

- **PSC** o *prescaler*: Un registro que sirve para dividir la frecuencia de entrada. Normalmente, la frecuencia del procesador están en el orden de MHz. El prescaler suele dividirlo al orden de KHz o incluso menos, según las necesidades de temporización.
- **CNT** o *counter*: El registro que lleva la cuenta actual del timer. Suma 1 cada vez que la señal de reloj, dividida entre el factor del prescaler, se activa. Su frecuencia de actualización f_{CNT} es por tanto:

$$f_{CNT} = \frac{f_{CLK}}{\text{PSC} + 1}$$

- **ARR** o *auto-reload*: Valor máximo de conteo. Cuando **CNT** alcanza este valor, se resetea y genera un evento de interrupción que podremos controlar. El periodo de evento T_{UEV} es por tanto:

$$T_{UEV} = \frac{\text{ARR} + 1}{f_{CNT}} = \frac{(\text{PSC} + 1)(\text{ARR} + 1)}{f_{CLK}}$$

- **CCR** o *capture compare*: Registros que permiten diversas configuraciones de comparación con CNT (mayor que, menor, igual ...) para medir la duración de señales externas o generar formas de onda PWM, entre otras funcionalidades. En ocasiones también permiten interrupciones adicionales a la de **ARR**.

Podemos ver un diagrama de funcionamiento en la Figura 19.

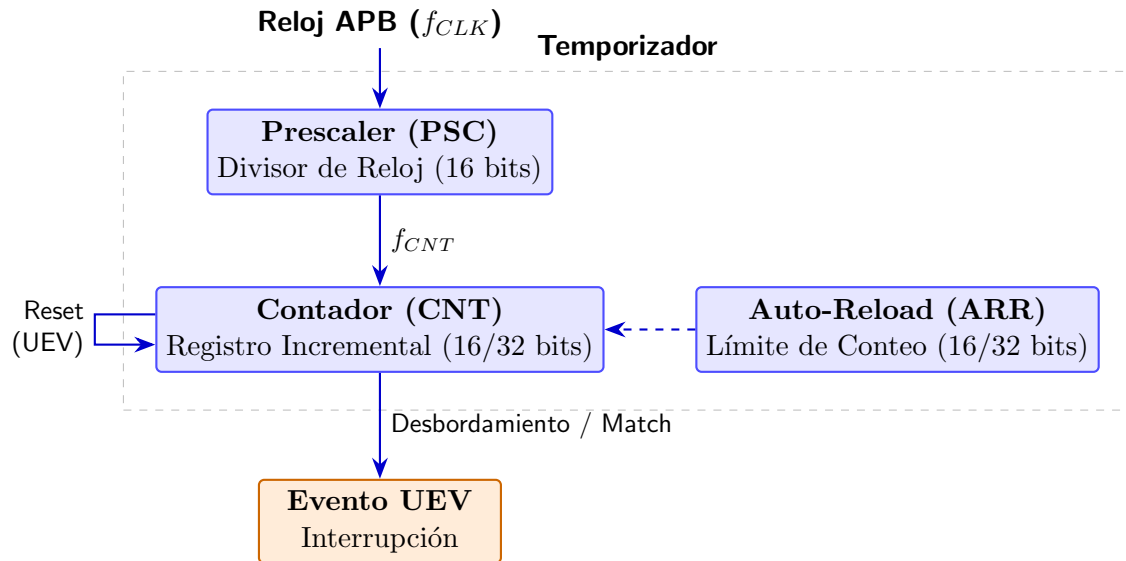


Figura 19: Diagrama de funcionamiento simplificado para los timers

6.3. Desarrollo de la práctica

Lo primero, como siempre, es ir añadiendo a nuestro `bsp.h` lo necesario para poder gestionar los timers. Vamos a utilizar dos, `TIM2` y `TIM3`. Consulta [rm0431 1.6](#) para ver las direcciones base de memoria, y [rm0431 19.4](#) para los registros de los temporizadores que permitan crear el `typedef` adecuado.

```

1 //... completar
2 #define TIM2_BASE_ADDR
3 #define TIM3_BASE_ADDR
4
5 //Structure for Timers (TIM2 / TIM3)
6 typedef struct
7 {
8     volatile uint32_t CR1;           /*!< TIM control register 1,
9         Address offset: 0x00 */
10    volatile uint32_t CR2;           /*!< TIM control register 2,
11        Address offset: 0x04 */
12    //... completar
13 } TIM_TypeDef;
14
15 #define TIM2 ((TIM_TypeDef *) TIM2_BASE_ADDR)
16 #define TIM3 ((TIM_TypeDef *) TIM3_BASE_ADDR)

```

A continuación tenemos que crear las funciones necesarias para el control de los timers:

```

1 //funciones de configuración
2 //... activa en CR1 el bit correspondiente para activar el contador
3 static inline void TIM_ENABLE_COUNTER(volatile TIM_TypeDef *TIMx, uint32_t
4     enable);
5 //... activa el modo oneshot desde CR1
6 static inline void TIM_ENABLE_ONESHOT(volatile TIM_TypeDef *TIMx, uint32_t
7     enable);
8 //... configura el registro PSC con el valor de preescalado
9 static inline void TIM_SET_PRESCALER(volatile TIM_TypeDef *TIMx, uint32_t value)
10 ;
11 //... configura el registro ARR con el valor de autoreload
12 static inline void TIM_SET_AUTO_RELOAD(volatile TIM_TypeDef *TIMx, uint32_t

```

```

    value);
10 //... activa las interrupciones en el registro DIER
11 static inline void TIM_ENABLE_INT(volatile TIM_TypeDef *TIMx);
12 //funciones de uso
13 //... borra el flag pendiente del registro SR
14 static inline void TIM_CLEAR_FLAG(volatile TIM_TypeDef *TIMx);
15 //... comprueba si hay flag pendiente en el registro SR
16 static inline uint32_t TIM_CHECK_FLAG(volatile TIM_TypeDef *TIMx);

```

También es necesario crear una función que nos permita inicializar el timer, llamando a las anteriores de configuración:

```

1 //congiura prescaler, autoreload, activa counter y oneshot si es necesario y
  activa interrupciones
2 static void TIM_INIT(TIM_TypeDef *TIMx, uint32_t prescaler, uint32_t autoreload
  , uint32_t enable_counter, uint32_t enable_oneshot);

```

Y por último tendremos que activar los bits correspondientes en **APB1ENR** ([rm0431 5.3.13](#)), y adicionalmente activar en **NVIC** ([rm0431 9.1.12](#)) las líneas de interrupción adecuadas:

```

1 RCC_APB1ENR->bits.TIM2EN = 1;
2 RCC_APB1ENR->bits.TIM3EN = 1;
3 TIM_INIT(TIM2, 15999, 500, 1, 0); //500 milisegundos, activo
4 TIM_INIT(TIM3, 15999, 10, 0, 1); //10 milisegundos, inactivo, oneshot
5 //Enable interrupts
6 NVIC_ENABLE_INT(TIM2_IRQn);
7 NVIC_ENABLE_INT(TIM3_IRQn);

```

Ahora podemos, por fin, hacer uso de las funciones de los timers para ver su efecto:

```

1 // Handles Task 1: Blink LED1 (Green) each 0.5 seconds
2 void TIM2_IRQHandler(void) {
3     //... completar comprobar que el flag está activo
4     if () {
5         //... completar borrar el flag
6         //... completar funcionalidad deseada
7     }
8 }
9
10 void TIM3_IRQHandler(void) {
11     //... completar comprobar que el flag está activo
12     if () {
13         //... completar borrar el flag
14         //... completar funcionalidad deseada
15     }
16 }

```

La funcionalidad que queremos se muestra en la Figura 20

- **TIM2** enciende o apaga el led verde (PB1) cada vez que se dispara. Para ello, asegúrate de haber activado dicho puerto y pin como **GPIO**.
- **TIM3** sirve como *debouncer* para el botón (en PA0). Para ello debemos conseguir que, al pulsarse el botón, se inhabilite durante el tiempo que tarde el timer en dispararse. Si tras ese tiempo el botón sigue pulsado, contaremos que es una pulsación de verdad. En caso contrario, lo omitiremos. En ambos casos, al terminar el temporizador volveremos a activar las interrupciones del botón. Como está en modo *oneshot*, cada vez que lo activemos se disparará solo una vez, evitando problemas.

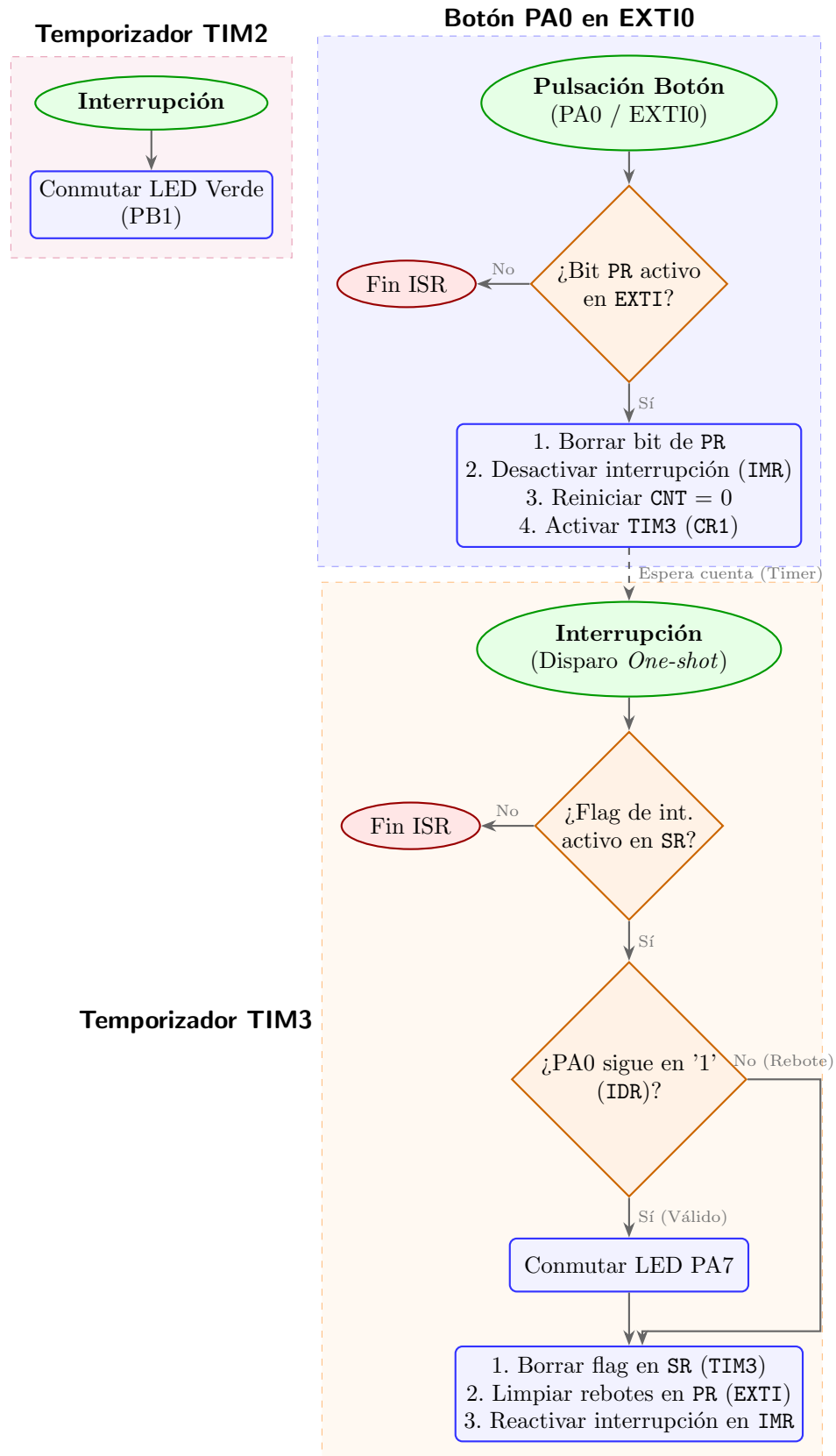


Figura 20: Diagrama de flujo funcional de las interrupciones.

Para el botón, debemos comprobar tras la activación que el bit correspondiente de **PR** está activo en **EXTI** ([rm0431 10.9.6](#)). Si es así, lo borramos y a continuación desactivamos la interrupción desde el registro **IMR** ([rm0431 10.9.1](#)). Finalmente activamos el temporizador, reseteando **CNT** a cero y activándolo desde **CR1** ([rm0431 19.4.1](#)).

```

1 void EXTI0_IRQHandler(void)
2 {
3     //... comprobar si la interrupción está pendiente
4     if () {
5         //... borrar bit pendiente
6         //... desactivar interrupciones de botón
7         //... resetear cuenta de timer
8         //... activar timer
9     }
10 }

```

En cuanto al timer, debemos en primer lugar comprobar y borrar si es necesario el flag de interrupción desde **SR** ([rm0431 19.4.5](#)). A continuación, y si el botón sigue pulsado (podemos comprobarlo desde el registro **IDR** del **GPIO** ([rm0431 6.4.5](#))), efectuar la función deseada (por ejemplo hacer parpadear el LED conectado a PA7). Tras ello, borraremos interrupciones pendientes del botón (por si se hubiera quedado pillado con rebotes) desde **PR** de **EXTI** ([rm0431 10.9.6](#)), y re-activaremos las interrupciones de botón en **IMR** ([rm0431 10.9.1](#)). Una cuestión importante es que, gracias a haber configurado el temporizador en modo *oneshot*, no es necesario desactivar sus propias interrupciones ya que no volverá a dispararse hasta reiniciar la cuenta.

```

1 // Debounce Lockout Timer Handler (Fires 10ms after initial edge)
2 void TIM3_IRQHandler(void) {
3     //... comprobar flag de interrupción
4     if () {
5         //... borrar flag de interrupción
6         //... verificar que sigue pulsado PA0
7         if () {
8             //... funcionalidad... Toggle led de PA7
9         }
10        //... borrar interrupciones pendientes para EXTI0 (botón)
11        //... reactivar interrupciones para EXTI0
12    }
13 }

```

6.4. Para hacer más

Hay un “temporizador” especial llamado **RTC** (*Real Time Clock*) capaz de llevar la cuenta de la hora y fecha actual de forma precisa. Con ello, además, puede generar interrupciones a largo plazo. Se utiliza principalmente para realizar esperas largas donde el procesador está dormido mientras espera la interrupción. Consulta la documentación y utilízalo para hacer parpadear el led azul (PA5) cada 5 segundos.

7. Práctica 6: Conexión con periféricos externos - Display y drivers

7.1. Objetivos

- Realizar conexión con periféricos externos, en concreto el display.
- Programar un driver sencillo para controlar el pintado.

7.2. Fundamentos teóricos

Cualquier procesador, sea del tipo que sea, se acompaña siempre, además de los periféricos internos, de periféricos externos. De lo contrario, la interfaz de comunicación con el usuario resultaría imposible. Desde simples pines con los que comunicar vía **UART**, como ya hemos visto, hasta sistemas complejos como los que veremos en esta práctica. Las conexiones de las que dispone nuestra placa **32F723EDISCOVERY** se pueden observar en [um2140 5](#), aunque las reproducimos en la Figura 21 para facilitar la consulta.

En concreto, observando dicha imagen, vemos que el display (a la izquierda) se conecta con la placa a través de GPIOs, y un periférico interno conocido como **FMC**. El **FMC** o *Flexible Memory Controller* ([rm0431 12](#)) es un periférico que permite gestionar la conexión con memorias externas de diferentes tipos (SDRAM, NAND flash, PSRAM, NOR flash...) además de dispositivos basados en memorias, como los displays (que guardan la imagen de manera similar a como una memoria guarda datos). Esto libera la memoria interna del **STM32F723IE** para poder guardar programas, en lugar de pesadas imágenes.

Dichas conexiones requieren el manejo simultáneo de múltiples pines de entrada y salida, lo cual sería imposible de realizar directamente con el procesador. Por tanto, este periférico **FMC** permite, con una configuración apropiada, gestionar las transferencias de datos de manera autónoma. Podemos efectivamente comprobar que efectivamente el display está conectado al **FMC** observando [um2140 5.14](#). Adicionalmente, en [um2140 6.7](#), nos dice qué pines se utilizan para las conexiones. (Estos nombres no son directamente los puertos GPIO de la placa, que se pueden consultar en [um2140 Table 19](#).) Cruzando estos datos con [rm0431 12.3](#), podemos ver qué señales concretas se utilizan para esta placa concreta.

Pero hay una peculiaridad, y es que el **FMC** en este caso no se comunica directamente con el display, sino que, de por medio, hay un chip controlador **ST7789H2**. Este chip está encargado de recibir comandos a través del **FMC**, configurando la memoria del display, así como sus características. Así pues, toda la información o comandos que queramos pasar al display deberán pasar por el **FMC**.

Adicionalmente, el propio panel LCD (modelo FRD154BP2901 de *Frida*) cuenta con unas conexiones especiales que no controla el **ST7789H2**. Estas conexiones, concretamente, son tres pines ([um2140 Table 14](#)): PH7 (resetea el panel), PH11 (se encarga de encender la luz trasera) y PC8 (Interrupción de Tear Effect, no hace falta utilizar este último).

Es decir, que parte del control lo haremos directamente, y parte a través del periférico **FMC**. Podemos ver todas estas conexiones de manera simplificada en Figura 22.

Ahora, con todas las conexiones hechas, hay que activar el panel. Para activar periféricos, se siguen ciertas secuencias de inicialización que ponen el hardware en un estado inicial. Aunque

Figure 3. Hardware block diagram

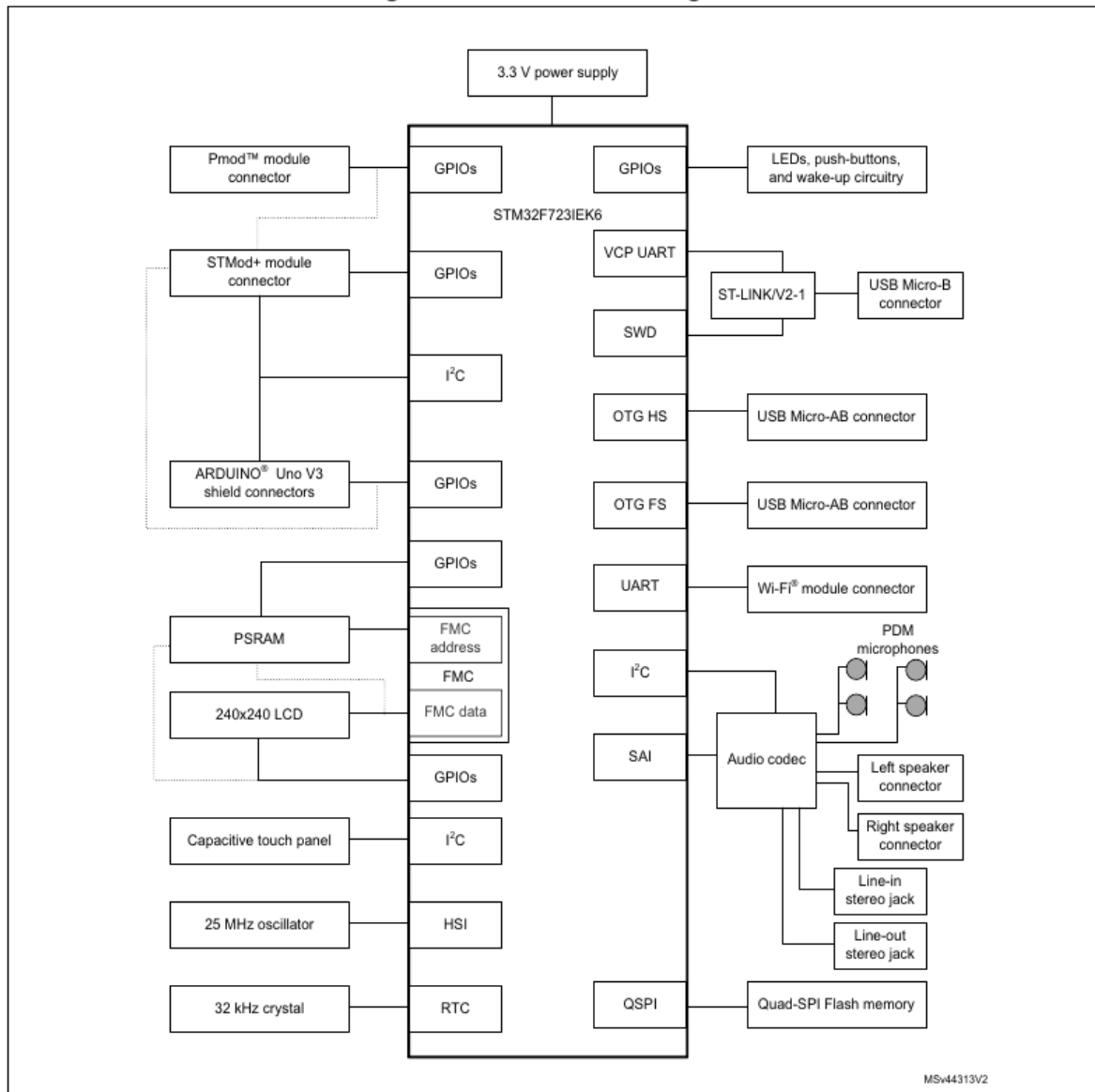


Figura 21: Diagrama de bloques de la placa **32F723EDISCOVERY**

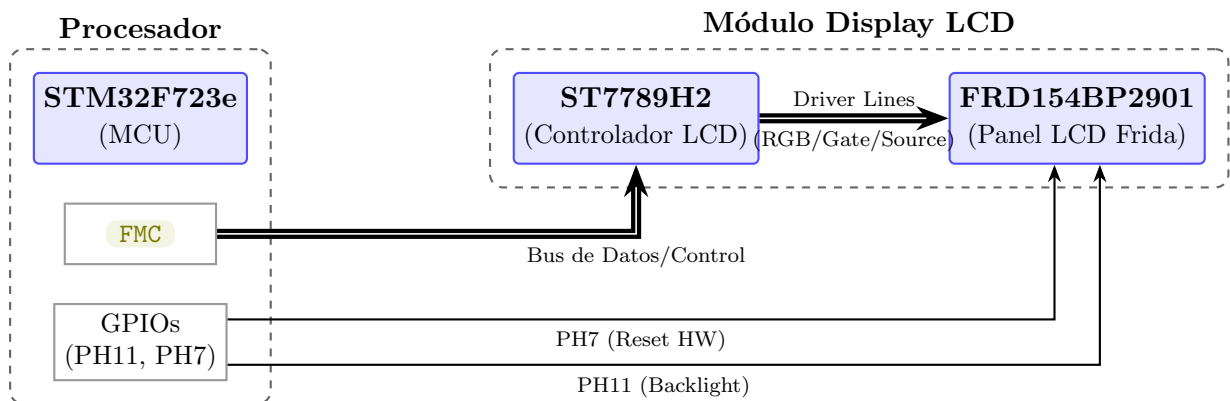


Figura 22: Diagrama de conexiones entre la MCU STM32F723, el controlador ST7789H2 y el panel LCD.

la documentación no es muy clara, si os fiáis de vuestro profesor, y no habéis abandonado la asignatura por dificultad (espero que no, si es así contactadme), el proceso no es excesivamente complicado. Habrá primero que activar el panel a través de los pines de conexión directa (básicamente, resetearlo), y luego activar el controlador [ST7789H2](#) utilizando comandos enviados por el [FMC](#). En concreto, la secuencia necesaria de encendido es:

1. Encender la luz trasera del display desde el pin PH11 (esto es opcional, pero queremos ver la pantalla, no?).
2. Aplicar la secuencia de reset en PH7: 0 (esperar 5 ms) → 1 (esperar 10 ms) → 0 (esperar 20 ms) → 1 (esperar 10 ms).
3. Con el panel encendido, toca el turno al controlador, al que tenemos que mandar un comando de **SWRESET**.
 - a) Mandar un comando de **SLEEP_IN** [ST7789H2 9.1.11](#) para asegurar que el panel está desactivado. La documentación obliga a esperar 5ms, esperamos 10ms para asegurar.
 - b) Mandar un comando de **SWRESET** [ST7789H2 9.1.2](#) para resetear el panel. La documentación obliga a esperar 120ms, esperamos 200ms para asegurar.
 - c) Mandar un comando de **SLEEP_OUT** [ST7789H2 9.1.12](#) para activar el panel. La documentación obliga a esperar 5ms, pero como menciona 120ms también, esperamos ese tiempo para asegurar.

Lo más interesante aquí es que estos comandos que debe recibir el panel LCD, se van a enviar a través del [FMC](#). Es decir, que estamos delegando la configuración en este periférico versátil, en lugar de tener que hacerlo nosotros a través de los aproximadamente 30 pines que lo controlan. Y ahora, cómo funciona todo esto?

7.2.1. Funcionamiento del periférico FMC

El periférico [FMC](#) funciona, a ojos del programador, como una extensión de la memoria RAM del sistema. En lugar de requerir un protocolo de comunicación complejo con llamadas a funciones de transmisión bit a bit, el [FMC](#) mapea el controlador externo directamente en el mapa de memoria del microcontrolador ([rm0431 12.4](#)).

Esto significa que escribir o leer en el controlador de pantalla es tan sencillo como hacer una operación de escritura en una dirección de memoria (*pointer dereferencing*) en C. El propio hardware del [FMC](#) se encarga de traducir esa escritura en las señales eléctricas apropiadas (bus de datos, activación de CS, pulsos de WR/RD, etc.) a través de los pines GPIO asignados. En este caso, únicamente se mapean dos direcciones: **REG** (para indicar el registro o comando del periférico) y **RAM** (para indicar el valor que se escribe):

```
1 typedef struct
2 {
3     volatile uint16_t REG;
4     volatile uint16_t RAM;
5 } FMC_CONTROLLER_TypeDef;
6
7 #define FMC_BANK2 ((FMC_CONTROLLER_TypeDef *) FMC_BANK2_BASE_ADDR)
8
9 static inline void FMC_BANK2_WriteReg(uint8_t Reg)
10 {
```

```

11     /* Write 16-bit Index, then write register */
12     FMC_BANK2->REG = Reg;
13     __DSB();
14 }
15
16 static inline void FMC_BANK2_WriteData(uint16_t Data)
17 {
18     /* Write 16-bit Reg */
19     FMC_BANK2->RAM = Data;
20     __DSB();
21 }

```

Para comunicarse con el **ST7789H2**, el **FMC** utiliza una interfaz de tipo 8080/6800 paralela de tipo SRAM. En esta interfaz, hay una línea de control crítica conocida como A0 (o Data/Command) ([um2140 Table 14](#)):

- Cuando la línea A0 está a nivel bajo (0), la transferencia se interpreta como un comando o instrucción para el controlador.
- Cuando la línea A0 está a nivel alto (1), la transferencia se interpreta como datos (parámetros del comando o colores de los píxeles).

Gracias al mapeo en memoria, en este caso en el banco 2 (BANK2) del **FMC**, diferenciar entre enviar un comando o un dato es tan simple como escribir en dos direcciones de memoria consecutivas dentro del rango asignado al **FMC**. La dirección base (REG) enviará la línea A0 a 0 (Comandos), mientras que la siguiente (RAM) enviará A0 a 1 (Datos).

7.2.2. Funcionamiento del controlador ST7789H2

El chip **ST7789H2** es el “cerebro” interno del panel LCD. Su función principal es mantener la imagen visible en la pantalla refrescando continuamente los píxeles físicos, liberando por completo al **STM32F723IE**.

Para lograr esto, el **ST7789H2** contiene una memoria RAM interna de 240×320 localizaciones de memoria, que se configura mediante secuencias de comandos y datos concretas a través del **FMC**. El flujo general de trabajo con este chip consta de tres etapas:

1. **Configuración e Inicialización:** Tras la secuencia de reset, se le envían comandos mediante el **FMC** para definir el formato de color (por ejemplo, RGB565 de 16 bits por píxel), orientación de la pantalla (lectura vertical u horizontal), etc.
2. **Definición de Ventana de Trabajo:** Antes de pintar, se envía un comando de dirección de columnas (CASET, [ST7789H2 9.1.20](#)) y de filas (RASET, [ST7789H2 9.1.21](#)). Esto le dice al **ST7789H2** en qué sub-región o rectángulo de la pantalla vamos a escribir.
3. **Escritura de Píxeles:** Una vez delimitada la zona, se envía el comando de escritura en memoria (RAMWR, [ST7789H2 9.1.22](#)). A partir de ese momento, cada dato enviado por el **FMC** corresponderá al color de un píxel (se pueden enviar varios en ráfaga). El propio controlador incrementa automáticamente la dirección de memoria interna píxel a píxel, llenando la ventana definida de izquierda a derecha y de arriba a abajo.

Es decir, el procesador escribe los colores a cada píxel en la memoria mediante el **FMC**, el **ST7789H2** los almacena en su RAM interna y sus circuitos de control (drivers (no confundir con driver software)) convierten esos valores digitales en las tensiones físicas que configuran el display FRD154BP2901. Un diagrama mostrando este funcionamiento se observa en la Figura 23.

1. Mapa Memoria MCU

2. Periférico FMC

3. Controlador ST7789H2

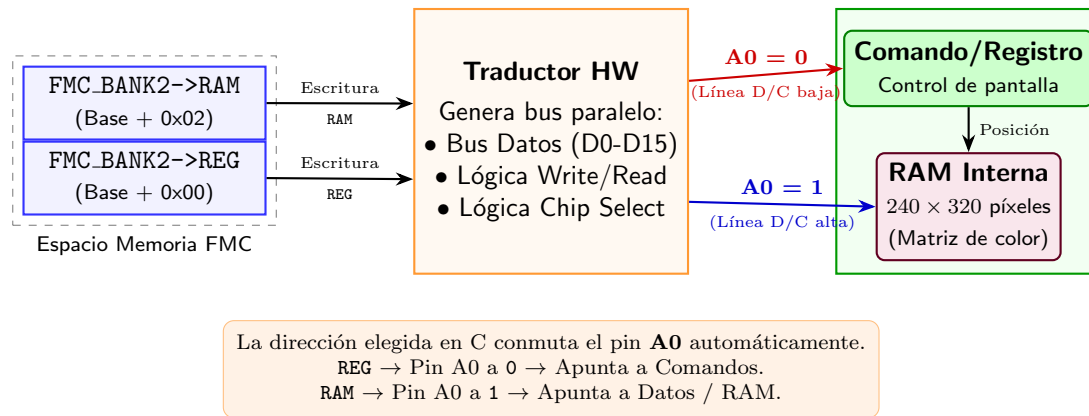


Figura 23: Mapeo de memoria a través del FMC hacia la lógica y RAM interna del ST7789H2

7.3. Desarrollo de la práctica

Para esta práctica, vamos a partir del código final de `main.c` al que queremos llegar. Básicamente, inicializaremos todos los elementos necesarios (el panel LCD, el periférico **FMC**, y el controlador **ST7789H2**), antes de dibujar algo sencillo en la pantalla. Se incluye además un pequeño código para poner píxeles aleatorios y ver que de verdad está funcionando:

```

1  // Definimos una variable global para la semilla (debe ser distinta de 0)
2  static uint32_t estado_rng = 123456789;
3
4  // Genera un entero sin signo de 32 bits (0 a 4,294,967,295)
5  uint32_t rand32(void) {
6      uint32_t x = estado_rng;
7      x ^= x << 13;
8      x ^= x >> 17;
9      x ^= x << 5;
10     return estado_rng = x;
11 }
12
13 int main(void)
14 {
15     LCD_Init();
16     FSMC_Init();
17     ST7789H2_INIT();
18
19     /* Clear the LCD */
20     for (int i = 0; i < 240; i++)
21         for (int j = 0; j < 240; j++)
22             ST7789H2_WritePixel(i, j, LCD_COLOR_WHITE);
23
24
25     for (int i = 50; i < 60; i++)
26         for (int j = 50; j < 60; j++)
27             ST7789H2_WritePixel(i, j, LCD_COLOR_RED);
28

```

```

29
30     while(1)
31     {
32         uint32_t rnd = rand32();
33         ST7789H2_WritePixel((rnd >> 24) & 0xff, (rnd >> 16) & 0xff, rnd & 0xffff
34         );
35     }

```

Yendo por orden, lo primero es inicializar el panel LCD. Lo primero es configurar los pines de conexión directa (de reset, interrupción (que no utilizaremos) y luz trasera). Posteriormente, aplicaremos la secuencia de inicialización del panel.

```

1  void LCD_Pins_Init() {
2      /* Initialize LCD special pins GPIOs */
3      /* LCD_RESET GPIO configuration */
4      RCC_AHB1ENR->bits.GPIOHEN = 1;
5      GPIO_CONFIG(GPIOH, 7, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
6      GPIO_PUPD_NONE, 0);
7      /* LCD_TE GPIO configuration */
8      RCC_AHB1ENR->bits.GPIOCEN = 1;
9      GPIO_CONFIG(GPIOC, 8, GPIO_MODE_INPUT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
10     GPIO_PUPD_NONE, 0);
11     /* LCD_BL_CTRL GPIO configuration */
12     RCC_AHB1ENR->bits.GPIOHEN = 1;
13     GPIO_CONFIG(GPIOH, 11, GPIO_MODE_OUTPUT, GPIO_OTYPE_PP, GPIO_OSPEED_LOW,
14     GPIO_PUPD_NONE, 0);
15 }
16
17 void LCD_Startup_Seq() {
18     /* Backlight enable */
19     GPIO_PIN_WRITE(GPIOH, 11, GPIO_STATE_ONE);
20
21     /* Apply hardware */
22     //... completar escribir en el pin de reset la secuencia de activación
23     //... utiliza la función DELAY para las esperas
24 }
25
26 void LCD_Init() {
27     LCD_Pins_Init();
28     LCD_Startup_Seq();
29 }

```

Una vez iniciado el panel, lo siguiente es activar el **FMC**. La configuración es bastante complicada y ya está hecha en el código proporcionado de base. Se recomienda echar un vistazo. En resumen, lo que hacemos es activar primero todos los pines requeridos por el controlador, y luego configurar el modo de conexión con la memoria, así como las características de temporización del bus.

Finalmente, se configura el controlador del display, enviando comandos directamente desde el **FMC**. El código se encuentra en `ST7789h2.h`, y hay que completar algunas partes para hacerlo funcionar completamente. En concreto:

En la función `ST7789H2_INIT`, al principio, debemos inicializar el controlador, mandando los comandos adecuados:

```

1  /* Sleep In Command */
2  ST7789H2_WriteReg(ST7789H2_SLEEP_IN, (uint8_t*)NULL, 0);
3  /* Wait for 10ms */
4  DELAY(10);

```

```
5
6 //... completar mandar el comando de RESET, seguido de una pausa de 200ms
7 //... completar mandar el comando de SLEEP_OUT, seguido de una pausa de 120ms
```

También deberemos crear la función `DISPLAY_ON` para encender el display:

```
1 void ST7789H2_DisplayOn(void)
2 {
3     //... completar mandar el comando DISPLAY_ON
4     //... completar mandar el comando SLEEP_OUT
5 }
```

Con esto ya deberíamos tener el display funcionando! El código debería borrar el display a blanco (tardará un poco), pintar un cuadrado rojo, y luego empezar a dibujar cuadrados de colores aleatoriamente.

Amplía ahora la librería con funciones de dibujo de líneas, rectángulos, círculos... Así podrás tener mucho más control (y hacer de manera más fácil) el dibujo en pantalla. También te puede servir para más tarde desarrollar aplicaciones con interfaces bien chulas.

7.4. Para hacer más

Explora la documentación del display para estudiar las maneras más rápidas de pintar la pantalla (por líneas en lugar de píxeles, por ejemplo). También puedes aumentar la velocidad del procesador cambiando su frecuencia de reloj de los 16MHz por defecto a más de 200MHz para conseguir mayor velocidad.

8. Práctica 7: Buses de Expansión - I2C y pantalla táctil

8.1. Objetivos

- Entender el funcionamiento básico del bus I2C.
- Programar un driver para la comunicación con la pantalla táctil a través de I2C.

8.2. Fundamentos teóricos

Los procesadores tienen, por regla general, un gran número de pines disponibles para su conexión con dispositivos externos (de hecho, cuentan con varios miles en los modelos más avanzados). Sin embargo, esta disponibilidad tiene sus limitaciones: en primer lugar, no todos los pines son capaces de realizar todas las funciones. En segundo lugar, el uso de pines concretos requiere un grado de planificación del sistema completo a nivel de placa base.

Mucha veces queremos la posibilidad de conectar diferentes dispositivos a nuestro diseño, pero no necesariamente que éstos existan desde el comienzo. Por ejemplo, un microcontrolador sencillo puede utilizarse para controlar diferentes tipos de sensores, actuadores, pantallas, controles y, a nivel de diseño, sólo podremos ofrecer unos pocos pines al usuario.

Para tener la posibilidad de conectar multitud de dispositivos a nuestro procesador, existe el concepto de bus: protocolos que utilizan un conjunto reducido de pines, y permiten, a través de unas pocas líneas de datos, la interconexión de varios (en ocasiones hasta cientos) dispositivos.

El **Bus I2C** (*Inter-Integrated Circuit*) es una de estas alternativas. Es de las más sencillas, y permite, a través de únicamente dos líneas, la conexión de hasta 1000 dispositivos. Los elementos más destacables del bus I2C pueden verse en la Figura 24 y son:

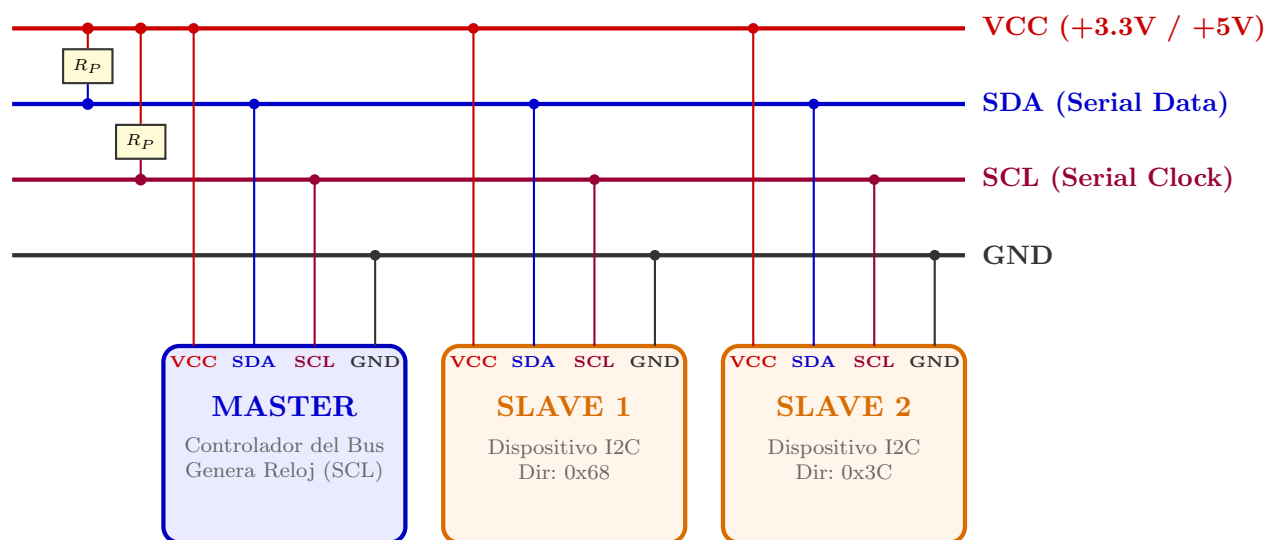


Figura 24: Diagrama de un bus I2C

- *Master*: Un único dispositivo que tiene el control del bus, y genera las peticiones en el mismo. El *Master* puede cambiar con ciertas señalizaciones especiales, pero siempre debe ser un único dispositivo.

- *Slave*: Pueden ser múltiples, son dispositivos que están conectados al bus y escuchan el mismo a la espera de peticiones del *Master*.
- **SDA** o *Serial Data*: Línea de datos serie, a través de la cual se envía la información (bien del *Master* al *Slave* o viceversa).
- **SCL** o *Serial Clock*: Línea de reloj serie. Controlada por el *Master*, lleva la señal de reloj con la que se sincronizan los datos de **SDA**. Generalmente, los dispositivos **I2C** permiten un amplio rango de frecuencias para su funcionamiento, aunque las más típicas son 100 o 400 KHz.

Un dato interesante a observar es el hecho de que tanto **SCL** como **SDA** tienen resistencia de *pull-up* a VCC. Es decir, que por defecto están siempre a “1”. Cualquier dispositivo que se conecte, operará en modo de *colector abierto* (open drain). Es decir, que únicamente podrá bajar la línea a “0” conectándola a tierra, enviándose los “unos” pasivamente.

8.2.1. Funcionamiento del bus I2C - capa física

Una vez realizadas las conexiones entre los dispositivos, la siguiente pregunta razonable es cómo opera el bus I2C. Las transferencias de datos siguen, de manera general, los siguientes pasos:

1. Una transferencia se inicia con una condición de inicio (**S** (start)), que se señala desde el *Master* bajando **SDA** a “0”, mientras **SCL** se mantiene a “1”.
2. A continuación **SCL** comienza a oscilar, bajando a “0” en primer lugar, mientras se prepara el bit a enviar en **SDA**.
3. Cuando **SCL** sube a 1, el bit enviado está disponible en **SDA** y el resto de dispositivos *Slaves* pueden leerlo.
4. Se repiten los pasos 2 y 3 tantas veces sea necesario para enviar todos los bits.
5. El último bit es seguido de un pulso de reloj sin dato, en el cual **SDA** se pone a “0”.
6. Finalmente, se indica la condición de fin (**P** (stop)), liberando primero **SCL**, y posteriormente **SDA**.

Este proceso se puede ver gráficamente en la Figura 25.

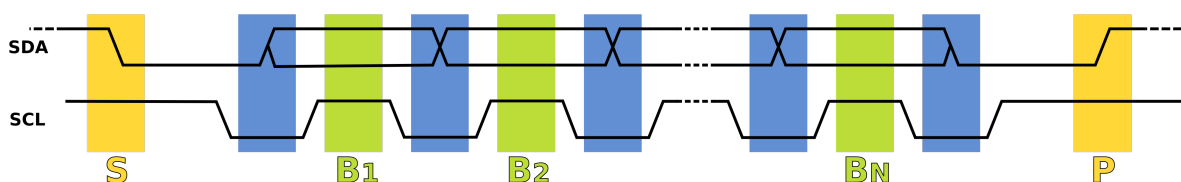


Figura 25: Transferencia de datos en I2C. (fuente:wikipedia)

8.2.2. Direccionamiento en I2C - capa lógica

Ahora bien, dentro de estas transferencias, ¿cómo sabemos a qué dispositivo van los bits? para ello, cada *Slave* cuenta con una dirección (de 7 bits, o 10 en versiones más modernas), habitualmente fijada por hardware, y con la que se puede identificar un dispositivo concreto. Así, para escribir, se seguirá el siguiente procedimiento:

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit “0” (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Envío del dato (8 bits). *Slave* responde **ACK**.
4. Generalmente, y según dispositivo, se pueden enviar datos consecutivos que irán a parar a registros consecutivos si se quiere.
5. Fin de comunicación con condición de parada **P**.

Para leer, el proceso es algo más complejo, ya que primero indicaremos el registro en modo escritura, y posteriormente cederemos el control al *Slave* en modo lectura para que nos pase el dato por el bus.

1. Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit “0” (write). *Slave* responde **ACK**.
2. Envío del comando (o registro destino) (8 bits). *Slave* responde **ACK**.
3. Re-Inicio de comunicación con condición inicial **S** seguida de la dirección (7 bits) + bit “1” (read). *Slave* responde **ACK**.
4. El *Slave* toma el control y envía el dato de vuelta (8 bits). El *Master* puede responder con **ACK** para pedir más datos consecutivos, o con **NACK** para terminar la comunicación.
5. Fin de comunicación con condición de parada **P**.

Las condiciones **ACK** y **NACK** se indican, respectivamente, con un “0” y un “1” (sí, están invertidas) por parte del receptor, en la línea **SDA** tras las transmisión de un byte.

El proceso para ambas transacciones puede observarse en la Figura 26

8.2.3. Líneas auxiliares a I2C

En ocasiones, utilizar únicamente las líneas **SCL** y **SDA** no es suficiente. Pensemos por ejemplo en el caso de las interrupciones. Con tan sólo las dos líneas de I2C, capturar una interrupción es imposible, tendríamos que hacer una encuesta activa a los periféricos, preguntando periódicamente si tienen algún dato de interés. Esto sería claramente ineficiente.

Es por ello que las interrupciones de los dispositivos sí van por líneas separadas, para que el procesador sea capaz de capturarlas, identificar directamente el dispositivo causante, y entonces,



Figura 26: Escrituras y lecturas, respectivamente, en I2C

y sólo entonces, encuestarlo vía I2C para recibir el dato. Es el caso, por ejemplo, con nuestra pantalla táctil y la detección de pulsaciones.

Adicionalmente, algunos dispositivos disponen de señales extra, para habilitarlos o resetearlos.

8.2.4. Conexiones en nuestra placa

Podemos ver el pinout completo de las conexiones procesador-pantalla táctil en [um2140 6.8](#). El conector a la pantalla táctil tiene señales **SCL**, **SDA**, **INT**, **RESET**, además de varias conexiones para alimentación y tierra. Estas últimas son pasivas y no requieren de control por parte del procesador.

Las que sí requieren de control, se especifican en [um2140 Appendix A](#). Ahí, podemos ver las conexiones que se especifican en la Tabla 1

Pin	Function	Connection
PA8	I2C3 SCL	CTP SCL
PH8	I2C3 SDA	CTP SDA
PH9	GPIO OUTPUT	CTP RST
PI9	GPIO EXTI9	CTP INT

Tabla 1: Conexiones para la pantalla. CTP: *Connector Touch Panel*. Los pines PA8 y PH8 no vienen completamente especificados en [um2140](#), pero se incluyen aquí por completitud.

8.3. Desarrollo de la práctica

Llegados a este punto, y con la teoría del bus I2C vista, sólo nos queda comunicarnos con el panel táctil. Haremos una aplicación donde el usuario “pintará” con el dedo en la pantalla. Para ello, partiremos de la práctica anterior, donde ya teníamos el controlador de la pantalla funcionando. En lugar de pintarse aleatoriamente, ahora controlaremos nosotros el pintado. Para ello, habrá que hacer lo siguiente:

- Configurar el bus I2C para poder comunicarnos con el panel táctil (CTP).

- Activar las interrupciones adecuadas para recibir eventos del CTP.
- Crear el driver que nos permita leer los eventos y transformarlos a coordenadas para pintar en la pantalla.

Vamos por orden:

8.3.1. Configuración del bus I2C

Lo primero que debemos hacer es activar los pines adecuados para la comunicación. En primer lugar, los pines **SCL** y **SDA** (PA8 y PH8) deben configurarse en modo alternativo con función 4 (esto los conecta al I2C3, ver [ds11853 Table 12](#)), modo open drain, sin pull up y velocidad alta. Además, hay que asegurarse de activar **GPIOAEN** y **GPIOBEN** en **AHB1ENR** ([rm0431 5.3.10](#)).

Por otro lado, debemos asegurarnos que el CTP no está en modo RESET, para ello, debemos activar el GPIO para el pin PH9 (Correspondiente a CTP RESET (Tabla 1)). Este pin debe configurarse en modo salida, push-pull sin pullup, y velocidad baja. A continuación debemos escribir un “1” en él para desactivar el RESET (que es en lógica inversa).

```

1 void I2C3_Pins_Init() {
2     //Pin A8 has I2C_SCL
3     // ... completar
4     //Pin H8 has I2C_SDA
5     // ... completar
6     //Pin H9 para el reset del CTP
7     // ... completar
8 }

```

Con esto los pines ya están activos, y nos toca configurar el periférico I2C. Las definiciones de los registros que contiene se pueden ver en [rm0431 26.9](#), y la dirección base se puede encontrar en [rm0431 1.6.2](#). Se ofrece a continuación el código para **bsp.h** con las definiciones y algunas funciones predefinidas creadas:

```

1 #define I2C3_BASE_ADDR 0x40005C00UL
2
3 typedef struct
4 {
5     volatile uint32_t CR1;        /* Control register 1,          */
6     volatile uint32_t CR2;        /* Control register 2,          */
7     volatile uint32_t OAR1;       /* Own address 1 register,      */
8     volatile uint32_t OAR2;       /* Own address 2 register,      */
9     volatile uint32_t TIMINGR;    /* Timing register,             */
10    volatile uint32_t TIMEOUTR;    /* Timeout register,            */
11    volatile uint32_t ISR;         /* Interrupt and status register, */
12    volatile uint32_t ICR;         /* Interrupt clear register,     */
13    volatile uint32_t PECR;        /* PEC register,                */
14    volatile uint32_t RXDR;        /* Receive data register,       */
15    volatile uint32_t TXDR;        /* Transmit data register,      */
16 } I2C_TypeDef;
17
18 #define I2C3 ((I2C_TypeDef *) I2C3_BASE_ADDR)
19
20 static inline void I2C_Disable(I2C_TypeDef * I2Cx) {
21     I2Cx->CR1 &= ~(0x1);
22 }
23

```

```

24 static inline void I2C_Enable(I2C_TypeDef * I2Cx) {
25     I2Cx->CR1 |= (0x1);
26 }
27
28 static inline void I2C_Set_Timing(I2C_TypeDef * I2Cx, uint32_t presc, uint32_t
    sclldel, uint32_t sdadel, uint32_t sclh, uint32_t scll) {
29     I2Cx->TIMINGR = (presc << 28) | (sclldel << 20) | (sdadel << 16) | (sclh <<
        8) | (scll);
30 }

```

En lo que a nosotros respecta, debemos utilizar estas funciones para inicializar el periférico **I2C3** ([rm0431 26](#)). En [rm0431 26.4.5](#), tenemos un diagrama que nos indica cómo proceder para configurar el controlador I2C. Como no necesitamos todas las funciones, lo haremos algo más sencillo.

Lo primero será activar el periférico **I2C3** desde **APB1ENR** ([rm0431 5.3.13](#)), para a continuación aplicar una secuencia de reseteo desde **APB1RST** ([rm0431 5.3.8](#)) consistente en escribir un 1 y luego un 0 sobre el bit de **I2C3**. Tras esto, ya podemos configurar el periférico. Para ello se inhabilita (ya desde dentro de su registro **I2C_CR1**), se configura el temporizador **I2C_TIMINGR**, y se vuelve a habilitar.

Para la temporización, hay numerosos campos. Tenemos que configurarlos teniendo en cuenta que la frecuencia base son 16MHz. La frecuencia objetivo que queremos son 400KHz. Hay muchas maneras de conseguirla, tenemos que tener en cuenta que la fórmula es la siguiente:

$$f_{obj} = \frac{f_{base}}{(PRESC + 1) * (SCLL + 1 + SCLH + 1)}$$

SCLL y **SCLH** indican el tiempo que la señal de reloj pasará, respectivamente, a baja (low) y alta (high). Podemos ponerlos iguales para conseguir una señal cuadrada. El valor de **PRESC** es un preescaler que nos divide la señal de reloj por si fuera muy alta. En nuestro caso, podemos dejar el prescaler a 0, y poner los otros dos registros a 19, para conseguir una onda cuadrada exacta a 400KHz. Los valores de **SCLDEL** y **SDADEL** controlan los desfases entre las subidas y bajadas de **SCL** y **SDA**. En general son valores bastante flexibles, aunque sus detalles son complejos y pueden explorarse más en [rm0431 26.9.5](#) y [rm0431 26.4.5](#). Para nuestro caso, con 3 y 1 respectivamente debería funcionar.

```

1 void I2C3_Reg_Config() {
2     //Enable I2C3 and reset cycle
3     // ... completar
4
5     //Configure I2C
6     I2C_Disable(I2C3);
7     // Set Timing
8     // ... completar
9     I2C_Enable(I2C3);
10 }

```

8.3.2. Interrupciones del CTP

A continuación debemos activar las interrupciones del panel táctil. Como vimos en Tabla 1 tiene un pin dedicado para interrumpir al procesador, a través de la línea de interrupción externa **EXTI9**. Debemos configurar el procesador para atender esa petición. Recordemos que cada

línea de interrupción externa `EXTIx` puede conectarse a cualquiera de los puertos (PA, PB, ...) `rm0431 10`. En este caso, como interrumpimos desde `PI9`, debemos configurar `EXTI9` para que atienda al puerto I en modo falling edge. Además, debemos indicar desde `SYSCFG` en el registro `EXTICR` (`rm0431 7.2.3`) que el puerto I (8) es el que se atiende en `EXTI9`, y activar `SYSCFG` desde el bit de enable de `APB2ENR` (`rm0431 5.3.19`). Finalmente, tenemos que activar la interrupción correspondiente en el `NVIC` (`rm0431 9.1.2`).

Hay que tener en cuenta que las interrupciones externas están agrupadas (en concreto, de la 5 a la 9 se atienden juntas (`rm0431 9.1.2`) a través de la línea de interrupción número 23 del procesador). Así que, en nuestra rutina de interrupción, tendremos que asegurarnos de que fue la interrupción externa 9 la que causó la interrupción a través del registro `PR` de `EXTI` (`rm0431 10.9.6`)

```

1 void TS_INT_Init(void) {
2     // ... completar activar GPIO PI9 como entrada pull-up, velocidad rápida
3
4     /* Enable and Map EXTI Line 9 to GPIOI (0b1000 = Port I) */
5     // ... completar activar SYSCFGEN en APB2ENR
6     // ... configurar int 9 para atenderse desde PI en EXTICR
7     // ... activar interrupción EXTI9 en EXTI
8     // ... habilitar modo falling edge y deshabilitar rising edge
9     // ... activar interrupción 23 (EXTI9_5) en NVIC
10 }
11
12 void EXTI9_5_IRQHandler(void) {
13     // ... completar comprobar bit 9 de EXTI PR
14     if () {
15         // ... limpiar bit 9 de EXTI PR
16         g_ts_interrupt_flag = 1;    // Notify main thread
17     }
18 }

```

Con esto, ya tendremos las interrupciones disparando ante eventos del panel táctil. De hecho, puedes probar a ejecutar el código con un *breakpoint* en la interrupción, y deberías ver que se dispara al tocar la pantalla, aunque no recojamos dato alguno.

8.3.3. Driver del I2C

Nos falta leer los datos correspondientes a la posición, y hacer algo con ellos. Para ello necesitamos dos cosas:

- Un *driver* para el periférico `I2C` que nos permita leer/escribir a través del bus (programaremos ambas funciones, aunque sólo utilizaremos la primera).
- Otro *driver* que, utilizando el primero, se comuniqué con el panel táctil para leer la posición de la pulsación.

Lo primero de todo es crear el driver para `I2C` (`rm0431 26`). Vamos a empezar por unas cuantas funciones auxiliares para el control de los diversos registros internos. Analízalas bien, viendo los registros que utilizan y el manual, para comprender su funcionamiento:

```

1 static inline void I2C_Write(I2C_TypeDef *I2Cx, uint8_t data) {
2     // Wait for TXDR to be empty (TXIS == 0)
3     while (!(I2Cx->ISR & (1U << 1U)));

```

```

4      // Load byte into Transmit Data Register
5      I2Cx->TXDR = data;
6  }
7
8  static inline uint8_t I2C_Read(I2C_TypeDef *I2Cx) {
9      // Wait for RXNE (Receive Buffer Not Empty flag)
10     while (!(I2Cx->ISR & (1U << 2U)));
11     // Read byte from Receive Data Register
12     return (uint8_t)(I2Cx->RXDR & 0xFFU);
13 }
14
15 static inline void I2C_Wait_TC(I2C_TypeDef *I2Cx) {
16     // Wait for TC (Transfer Complete flag)
17     while (!(I2Cx->ISR & (1U << 6U)));
18 }
19
20 static inline void I2C_Stop(I2C_TypeDef *I2Cx) {
21     // Generate STOP condition (STOP bit 14 in CR2)
22     I2Cx->CR2 |= (1U << 14U);
23     // Wait for STOPF (Stop detection flag bit 5 in ISR)
24     while (!(I2Cx->ISR & (1U << 5U)));
25     // Clear STOPF flag by writing to ICR (STOPCF bit 5 in ICR)
26     I2Cx->ICR |= (1U << 5U);
27 }

```

Uno de los registros más importantes es **CR2** ([rm0431 26.9.2](#)). Nos va a permitir configurar la transferencia que queremos realizar (dirección destino, número de bytes a transferir, si es escritura o lectura, y algunas funciones más que no utilizaremos). Estúdialo bien, e implementa la función `I2C_Start`:

```

1  typedef enum {
2      I2C_WRITE = 0U,
3      I2C_READ  = 1U
4  } I2C_Direction;
5
6  static inline void I2C_Start(I2C_TypeDef *I2Cx, uint8_t slave_addr,
7      I2C_Direction direction, uint32_t len) {
8      uint32_t cr2 = 0;
9
10     // ... completar Set 7-bit Slave Address (SADD[7:1])
11     // ... completar Set Transfer Length (NBYTES[23:16])
12     // ... completar Set Direction (RD_WRN bit 10: 0 = Write, 1 = Read)
13     // ... completar Generate START Condition (START bit 13)
14
15     // Apply configuration to hardware
16     I2Cx->CR2 = cr2;
17 }

```

Una vez tenemos estas funciones auxiliares, ya podemos programar la escritura y lectura a periféricos a través del bus I2C. Recordemos de Apartado 8.2.2 el funcionamiento general de I2C. Más allá de no necesitar controlar los ACK (ya que se generarán solos), debemos programar el resto de la lógica siguiendo los pasos allí indicados. Se proporciona el esqueleto a continuación:

```

1  /* Writes 1 byte to a specific register inside the sensor */
2  void I2C_Reg_Write(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg, uint8_t
3      val) {
4      //... completar Issue START + set up Address (Write Mode, 2 bytes)
5      //... completar Write Target Register Address
6      //... completar Write Value
7      //... completar Wait for transfer complete (TC)
8      //... completar issue STOP condition

```

```

8 }
9
10 /* Reads 'len' sequential bytes starting from 'reg' */
11 void I2C_Reg_Read_Seq(I2C_TypeDef *I2Cx, uint8_t slave_addr, uint8_t reg,
12     uint8_t *dest, uint32_t len) {
13     // ===== PHASE 1: WRITE REGISTER ADDRESS =====
14     // ... completar Issue START + set up Address (Write Mode, 1 Byte)
15     // ... completar Write target register address
16     // ... completar Wait for Transfer complete (TC)
17
18     // ===== PHASE 2: READ DATA PAYLOAD =====
19     // ... completar Issue START + set up Address (Read Mode, 'len' Bytes)
20     // ... completar Read 'len' bytes from the sensor
21     // ... completar Wait for Transfer complete (TC)
22     // ... completar issue STOP condition
23 }

```

8.3.4. Driver del CTP

Finalmente, y con las funciones **I2C** creadas, estamos listos para leer los registros del panel táctil con la información correspondiente a las coordenadas de la pulsación. La documentación del mapa de registros interno del panel, y sus valores es bastante escasa. Pero sí disponemos del driver oficial de ST en <https://github.com/STMicroelectronics/stm32-ft3x67/tree/main>. Consultando el fichero `ft3x67.h` podemos ver el mapa de registros, así como los diversos códigos de control y respuesta. Para simplificar el proceso y no hacer un driver tan complejo, únicamente vamos a fijarnos en los siguientes registros internos mostrados en la Tabla 2 y Tabla 3.

Addr	Register Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
0x02	FT3X67_TD_STAT_REG	Reserved [7:4]				Touch Points [3:0] (0–2)			
0x03	FT3X67_P1_XH_REG	Event [7:6]		Res [5:4]		P1_X[11:8] (MSB)			
0x04	FT3X67_P1_XL_REG	P1_X[7:0] (LSB)							
0x05	FT3X67_P1_YH_REG	Reserved [7:4]				P1_Y[11:8] (MSB)			
0x06	FT3X67_P1_YL_REG	P1_Y[7:0] (LSB)							
0x07	FT3X67_P1_WEIGHT_REG	P1_WEIGHT[7:0]							
0x08	FT3X67_P1_MISC_REG	P1_MISC[7:0]							
0x09	FT3X67_P2_XH_REG	Event [7:6]		Res [5:4]		P2_X[11:8] (MSB)			
0x0A	FT3X67_P2_XL_REG	P2_X[7:0] (LSB)							
0x0B	FT3X67_P2_YH_REG	Reserved [7:4]				P2_Y[11:8] (MSB)			
0x0C	FT3X67_P2_YL_REG	P2_Y[7:0] (LSB)							
0x0D	FT3X67_P2_WEIGHT_REG	P2_WEIGHT[7:0]							
0x0E	FT3X67_P2_MISC_REG	P2_MISC[7:0]							

Tabla 2: Disposición de los registros en el controlador de panel táctil FT3X67

Como de momento sólo nos interesa leer una coordenada, podemos leer los primeros 5 registros y será suficiente. Debemos crear un nuevo fichero llamado `FT3x67.h` que contenga la lógica para hacerlo:

Field / Mask	Bits	Name	Values / Description
TD_STAT_MASK	[3:0]	Active Touch Points	Number of active touch points detected (0 to 2).
TOUCH_EVT_FLAG	[7:6]	Event Flag	0x00 (00b): Press Down 0x01 (01b): Lift Up 0x02 (10b): Contact 0x03 (11b): No Event
POS_MSB_MASK	[3:0]	Position MSB	Upper 4 bits of X or Y coordinate (bits [11:8]).
XL / YL	[7:0]	Position LSB	Lower 8 bits of X or Y coordinate (bits [7:0]).
WEIGHT	[7:0]	Touch Weight	Pressure / weight value of touch contact.
MISC	[7:0]	Miscellaneous	Touch area / extra touch panel metrics.

Tabla 3: Definiciones y valores para los registros del FT3X67

```

1  /*
2  * FT3X67.h
3  *
4  *   Created on: Aug 11, 2026
5  *   Author: dani
6  */
7
8  #ifndef FT3X67_H_
9  #define FT3X67_H_
10
11 #define TS_I2C_7BIT_ADDR 0x38
12 #define FT_REG_TD_STATUS 0x02
13
14 typedef struct {
15     uint16_t x;
16     uint16_t y;
17     uint8_t touch_detected;
18 } TS_State_t;
19
20 /**
21 * @brief Reads the current touch coordinates from the FT3X67.
22 * @param ts: Pointer to TS_State_t structure to store parsed coordinates
23 * @return 1 if touch is valid, 0 otherwise
24 */
25 uint8_t FT3X67_GetTouchCoordinates(TS_State_t *ts) {
26     uint8_t buffer[5];
27
28     /* Read 5 consecutive registers starting at TD_STATUS (0x02 to 0x06) */
29     // ... completar leer utilizando I2C_Reg_Read_Seq 5 bytes
30
31     uint8_t num_touches = buffer[0] & 0x0F;
32
33     if (num_touches > 0 && num_touches <= 2) {
34         // ... completar parsear coordenadas en ts->x y ts->y
35
36         ts->touch_detected = 1;
37         return 1;
38     }
39
40     ts->touch_detected = 0;
41     return 0;
42 }

```

```
43
44 #endif /* FT3X67_H_ */
```

8.3.5. Integración

Finalmente, ya estamos listos para integrar todo en nuestro código `main.c`. Tras inicializar el I2C y el panel táctil con las funciones `I2C3_Init()` y `TS_INT_Init()` podremos entrar en un bucle infinito que espere el aviso de interrupción del panel táctil antes de leerlo:

```
1 // ... código de inicialización de la pantalla (práctica anterior)
2
3 I2C3_Init();
4 TS_INT_Init();
5
6 while (1) {
7     /* Check if the touchscreen interrupt fired */
8     if (g_ts_interrupt_flag) {
9         g_ts_interrupt_flag = 0; // Reset software flag
10
11         // ... completar obtener coordenada utilizando
12         // ... el driver FT3X67 y pintarla en la pantalla
13         // ... hay que rotarla pues las coordenadas
14         // ... del panel y la pantalla no coinciden
15     }
16 }
```

8.4. Para hacer más

Investiga el repositorio oficial <https://github.com/STMicroelectronics/stm32-ft3x67/tree/main> para el driver del panel táctil, e intenta implementar detección de gestos. También puedes extender la lógica de tu programa para que, en lugar de pintar sólo los puntos que tocas, vaya haciendo una línea entre ellos (verás que, de lo contrario, los puntos no son perfectamente contiguos).

9. Práctica 8: *Bootloaders* y actualizaciones automáticas

9.1. Objetivos

- Entender el concepto de *bootloader* y programar uno.
- Aprender a comunicar con un módulo WiFi externo a la placa para la descarga de aplicaciones.
- Comprender los pasos a seguir para ceder el control de una aplicación a otra.
- Aprender cómo compilar las aplicaciones para que convivan con el *bootloader*.

9.2. Fundamentos teóricos

Hacer aplicaciones perfectas a la primera es algo muy complicado. Aún habiéndolas hecho, normalmente se requieren actualizaciones a lo largo del tiempo. Una vez hemos fabricado un dispositivo y está operativo, el enlace de depuración se suele perder, así que se requieren métodos más sofisticados para reprogramarlo. Uno de los más extendidos es el de programar un pequeño *bootloader*, o cargador, que se encarga además de actualizar el código principal si es necesario. En esta práctica programaremos un *bootloader* capaz de descargar aplicaciones via Wifi, grabarlas en la Flash, y por último cederles el control. Aprenderemos además cómo tenemos que compilar nuestras aplicaciones para que el *bootloader* las pueda cargar.

El *bootloader* es el primer programa que comienza a ejecutarse dentro del procesador. Su tarea, habitualmente, es realizar ciertas configuraciones iniciales, para posteriormente ceder el control a otro programa. Por ejemplo, en un ordenador “estándar”, el *bootloader* suele comprobar que los diferentes dispositivos están conectados correctamente y funcionando (tarjetas gráficas, memorias, pantalla, teclado, discos, etc), para posteriormente buscar distintos medios de arranque (USB, CD, Disco duro) y elegir automáticamente, o dar al usuario la opción, el modo de arranque del sistema.

Es desde este *bootloader* que uno puede configurar ciertos parámetros de muy bajo nivel del ordenador. Por ejemplo, encriptación de los discos (previa a la ejecución del sistema operativo), encendido automático a través de red, velocidad de los ventiladores...

Una vez se han realizado todas estas configuraciones y comprobaciones, el *bootloader* cede el control, generalmente, al sistema operativo (aunque en ocasiones hay varios niveles de *bootloading*). Éste ya gestionará el resto de tareas para su ejecución simultánea, gestionando los recursos del sistema.

En nuestro caso, no hay sistema operativo (aún), así que el *bootloader* simplemente cargará un programa sencillo. Para hacer algo más cercano a la realidad, haremos que nuestro *bootloader*, previo a la carga del programa, lo descargue automáticamente de un servidor si es que no estuviera ya en la memoria del procesador. Así imitaremos, en cierta forma, la capacidad de muchos dispositivos empotrados de actualizarse automáticamente, sin necesidad de ser configurados por los puertos de depuración.

Comprobará si la aplicación está ya instalada. De ser así, la lanzará directamente. Si no está, intentará descargarla de un servidor que configuraremos, por ejemplo, en nuestro teléfono. También daremos la opción de borrar la memoria donde alojaremos el programa, para poder

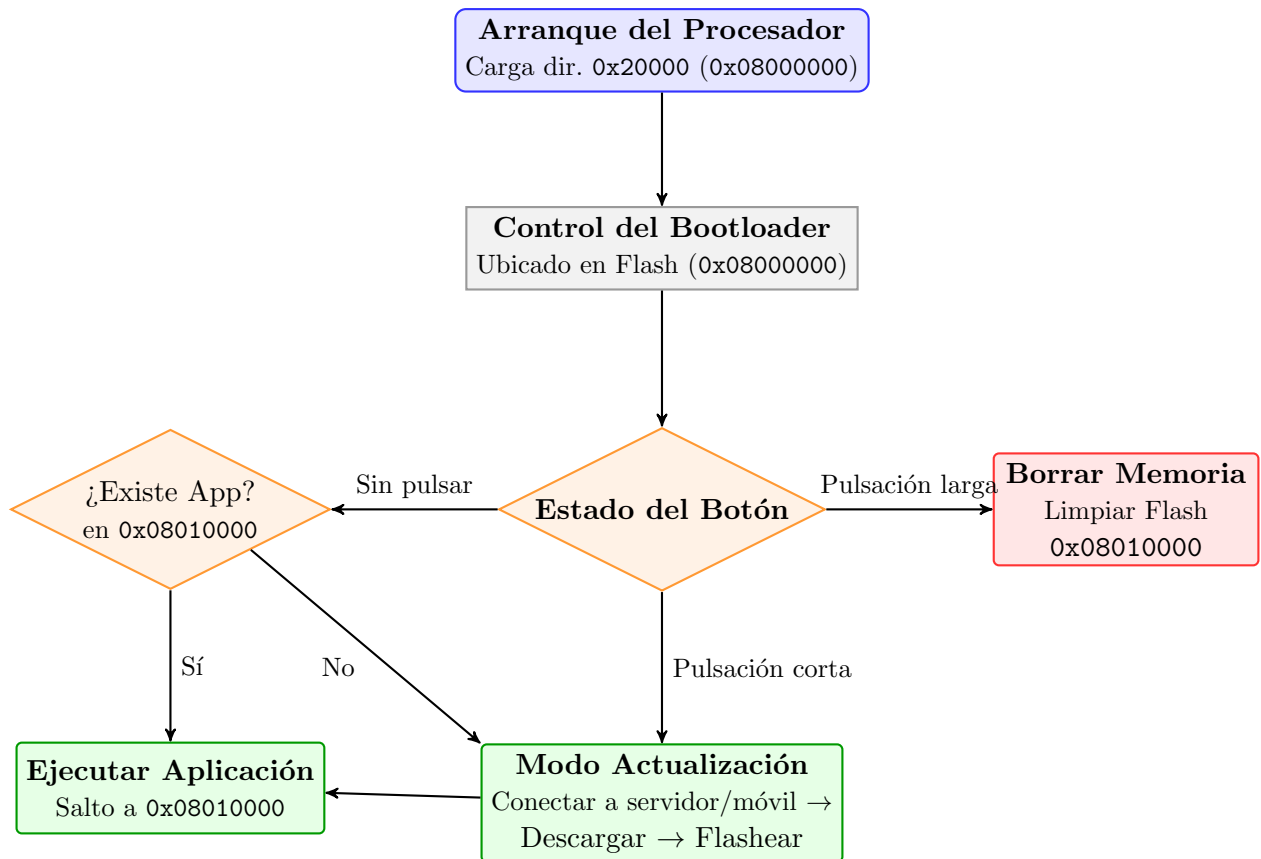


Figura 27: Secuencia de arranque de nuestra aplicación

probar el proceso varias veces sin necesidad de reprogramar la placa. El flujo de ejecución será el siguiente:

1. El procesador, desde su memoria de configuración, carga la dirección de arranque, en este caso la `0x20000` [rm0431 1.9](#). Nótese que esa dirección es un alias a la que nosotros utilizamos `0x08000000` [rm0431 3.3.1](#), ya que la **FLASH** se puede acceder desde dos rangos de direcciones diferentes.¹
2. El *bootloader* se encuentra allí, situado al principio de la flash `0x08000000` (memoria de arranque) y toma el control.
3. Si no hacemos nada, el *bootloader* consulta la dirección objetivo del programa (`0x08010000`) y verifica si existe un programa. En ese caso salta a ejecutarlo.
4. Si pulsamos un botón, el *bootloader* entrará en modo actualización, e intentará conectarse a internet para descargar, flashear, y por último ejecutar nuestra aplicación.
5. Si lo pulsamos aún más tiempo, borrará el contenido de la flash en la dirección `0x8010000`.

Cuando un *bootloader* cede el control, no puede hacerlo de cualquier manera. Debe asegurarse de que entrega el procesador a la aplicación invitada en un estado inicial aceptable. Por ejemplo,

¹Existe adicionalmente la posibilidad de programar dos direcciones diferentes de arranque, y controlar con un botón cuál de las dos se utiliza. Por defecto, la dirección de arranque será la de nuestro programa (*bootloader* en este caso), y en la otra, se encuentra otro *bootloader* diferente preprogramado por ST [rm0431 1.9](#). Para ver más detalles del *bootloader* de ST consultar [an2606](#)

deberá desactivar interrupciones, configurar correctamente la pila, e indicar la nueva dirección de inicio de programa al procesador. Un fallo en cualquiera de estos pasos podría suponer que el programa cargado no llegue a funcionar, generalmente porque empiece a ejecutarse en zonas de memoria restringidas.

Por otro lado, las aplicaciones también deben estar preparadas para su correcta ejecución. Normalmente, al preparar una aplicación “usual” en un procesador *bare-metal* como el nuestro, la compilamos con las opciones por defecto y esto es suficiente, ya que se ejecuta desde la dirección inicial de ejecución del procesador. Pero en este caso, en esa dirección tenemos el *bootloader*. Por tanto, debemos compilarla con la dirección de inicio objetivo en la cual la vaya a cargar el *bootloader*, asegurando así un correcto funcionamiento.

9.2.1. El módulo Wifi ESP-01S

Nuestro procesador no tiene manera de realizar comunicación inalámbrica, por tanto se utiliza un módulo externo ESP-01S. Este módulo permite, a través de una comunicación **UART** desde el procesador **STM32F723IE**, establecer una conexión WiFi y enviar o recibir datos a través de protocolos como HTTP y TCP, que serán vistos desde nuestra aplicación a través de la **UART**.

Nosotros lo veremos solo como usuarios y con un conjunto reducido de comandos. Puedes consultar la documentación completa en https://espressif-docs.readthedocs-hosted.com/projects/esp-at/en/release-v2.2.0.0_esp8266/AT_Command_Set/index.html.

9.3. Desarrollo de la práctica

9.3.1. Configuración previa al *bootloader*

Como hemos visto anteriormente, la idea es hacer un *bootloader* con diversas opciones (descargar app, ejecutar, borrar) que se seleccionarán de manera semi-automática en función del estado de la memoria, y el de un botón que podremos pulsar más o menos tiempo.

Para medir el tiempo, vamos a hacer uso de un periférico que hasta ahora no habíamos utilizado, el **SysTick** **pm0253 4.4**. Este periférico se utiliza para llevar una cuenta de los *ticks* que han pasado en el procesador. La cuenta de *tick* se incrementa cada cierto tiempo, dependiente de la velocidad del reloj (16MHz por defecto) y la configuración del periférico. Por ejemplo, podemos configurarlo para que cuente cada milisegundo. Así, podremos consultar en cualquier momento el valor de *tick* actual para saber cuánto tiempo ha pasado desde otro momento en el que realizáramos otra consulta. El código necesario se incluye a continuación:

```
1 #define SYSTICK_BASE_ADDR      (0xE000E010UL)
2
3 typedef struct {
4     volatile uint32_t CTRL;      // 0x00
5     volatile uint32_t LOAD;      // 0x04
6     volatile uint32_t VAL;       // 0x08
7     volatile uint32_t CALIB;     // 0x0C
8 } SysTick_TypeDef;
9
10 #define SysTick                 ((SysTick_TypeDef *) SYSTICK_BASE_ADDR)
11
12 volatile uint32_t msTicks = 0;
13 void SysTick_Handler(void) {
```

```

14     msTicks++;
15 }
16
17 void SysTick_Init(void) {
18     // Default HSI clock is 16MHz on reset. 1ms tick = 16,000 cycles.
19     SysTick->LOAD = (16000U - 1U);
20     SysTick->VAL = 0U;
21     SysTick->CTRL = (1U << 2) | (1U << 1) | (1U << 0); // Processor Clock, Int
        Enable, Enable
22 }
23
24 void SysTick_Disable(void) {
25     SysTick->LOAD = 0U;
26     SysTick->VAL = 0U;
27     SysTick->CTRL = 0U;
28 }
29
30 uint32_t GetTick(void) {
31     return msTicks;
32 }
33
34 void Delay_ms(uint32_t ms) {
35     uint32_t start = GetTick();
36     while ((GetTick() - start) < ms);
37 }

```

Por otro lado, necesitamos configurar los periféricos a utilizar por nuestro bootloader. Estos son:

- El botón de usuario.
- La **UART** de depuración (ya configurada en prácticas anteriores)
- La **UART** de comunicación con el ESP-01S

Se proporcionan las siguientes funciones ya hechas para ello:

```

1 void Peripherals_Init(void) {
2     // Enable USARTs and their GPIOs
3     RCC_AHB1ENR->bits.GPIOCEN = 1;
4     RCC_AHB1ENR->bits.GPIODEN = 1;
5     RCC_APB1ENR->bits.USART5EN = 1; // comm with espressif
6     RCC_APB2ENR->bits.USART6EN = 1; // comm with computer (stlink)
7
8     /* --- USART6 Pin Configuration (ST-LINK VCP: PC6=TX, PC7=RX) --- */
9     GPIO_CONFIG(GPIOC, 6, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
10         GPIO_PUPD_PU, GPIO_AF8_USART6);
11     GPIO_CONFIG(GPIOC, 7, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
12         GPIO_PUPD_PU, GPIO_AF8_USART6);
13
14     /* --- UART5 Pin Configuration (ESP8266: PC12=TX, PD2=RX) --- */
15     GPIO_CONFIG(GPIOC, 12, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
16         GPIO_PUPD_PU, GPIO_AF8_USART5);
17     GPIO_CONFIG(GPIOD, 2, GPIO_MODE_ALT, GPIO_OTYPE_PP, GPIO_OSPEED_HIGH,
18         GPIO_PUPD_PU, GPIO_AF8_USART5);
19
20     /* --- Baud Rate Setup ---
21     * Default HSI Clock = 16 MHz. Target Baud Rate = 115200.
22     * USARTDIV = 16,000,000 / 115200 = 138.88 => BRR = 139 (0x8B)
23     */

```

```

20     UART_BRR_SET(USART6, 139);
21     UART_BRR_SET(USART5, 139);
22     // enable UARTS + TX + RX
23     UART_ENABLE_SET(USART6, 1);
24     UART_TX_MODE_SET(USART6, 1);
25     UART_RX_MODE_SET(USART6, 1);
26     UART_ENABLE_SET(USART5, 1);
27     UART_TX_MODE_SET(USART5, 1);
28     UART_RX_MODE_SET(USART5, 1);
29 }
30
31 void UserButton_Init(void) {
32     RCC_AHB1ENR->bits.GPIOAEN = 1;
33     GPIO_CONFIG(GPIOA, 0, GPIO_MODE_INPUT, GPIO_OTYPE_PP, GPIO_OSPEED_LOW,
34                 GPIO_PUPD_PD, 0);
35 }

```

9.3.2. Cesión del control a la aplicación

Para poder ceder el control a la aplicación, debemos hacer dos cosas. En primer lugar, ser capaces de comprobar que la aplicación existe y está descargada en la memoria. En segundo, cambiar el control del procesador a esa zona de memoria para su ejecución.

Comenzamos con la comprobación de existencia. Las aplicaciones, al compilarse, tienen una estructura muy concreta que se genera a partir del *Linker Script*. Este fichero (STM32F723IEKX_FLASH.ld), que viene creado ya de manera automática con nuestros proyectos (aunque podemos customizarlo), tiene esta pinta (se destacan aquí solo algunas partes):

```

1  // Punto de entrada del código al comenzar a ejecutarse
2  // Esta función llama luego a main
3  ENTRY(Reset_Handler)
4
5  // posición en memoria de la pila, justo al final de la memoria RAM
6  _estack = ORIGIN(RAM) + LENGTH(RAM);
7
8  // tamaños mínimos de pila y montón. Si no caben (por ser muy
9  // grande el programa), la compilación fallará
10 _Min_Heap_Size = 0x200;
11 _Min_Stack_Size = 0x400;
12
13 // direcciones, permisos y tamaño de las memorias
14 // importante que en la flash no se tienen permisos
15 // de escritura (por defecto)
16 MEMORY
17 {
18     RAM      (xrw)      : ORIGIN = 0x20000000,   LENGTH = 256K
19     FLASH    (rx)       : ORIGIN = 0x80000000,   LENGTH = 512K
20 }
21
22 // aquí comienzan a colocarse los ‘‘trozos’’ de programa
23 SECTIONS
24 {
25     // primero una tabla de vectores con diferentes direcciones
26     // para las distintas interrupciones del sistema
27     .isr_vector :
28     {
29         . = ALIGN(4);
30         KEEP(*(.isr_vector)) /* Startup code */
31         . = ALIGN(4);

```

```

32     } >FLASH
33
34     // luego se coloca el código
35     .text :
36     {
37         . = ALIGN(4);
38         *(.text)          /* .text sections (code) */
39
40         // continúa (ver archivo de referencia en cualquier proyecto)
41         // .....

```

Este linker script gobierna cómo el código se ordena y coloca en el binario final. Básicamente, va colocando todos los símbolos (por símbolo se entienden variables, funciones, etc) uno detrás de otro. Los símbolos se definen desde el código, al definir funciones o usar variables. Aunque nosotros no lo hacemos a mano, cada vez que definimos una función, se crean símbolos que comienzan por `.text`, o cada vez que definimos variables, símbolos que comienzan por `.data`. Eso es lo que coloca el linker posteriormente y de manera ordenada.

Ciertos símbolos especiales, como el código primero que se ejecuta, se definen desde el fichero `startup_stm32f723ieqx.s` (ya creado por defecto con cualquier proyecto). Aquí tenemos la función `Reset_Handler` que será la primera ejecutada al lanzar cualquier programa:

```

1  .section .text.Reset_Handler
2  .weak Reset_Handler
3  .type Reset_Handler, %function
4  Reset_Handler:
5  ldr    r0, =_estack
6  mov    sp, r0          /* set stack pointer */
7
8  // ... inicialización de diferentes secciones de memoria
9  // salto a la función principal.
10 bl main

```

Y cómo sabe el procesador que tiene que llegar aquí al iniciar? Miremos en otra zona del código de inicialización:

```

1  .section .isr_vector,"a",%progbits
2  .type g_pfnVectors, %object
3
4  g_pfnVectors:
5  .word _estack
6  .word Reset_Handler
7  // ... punteros a interrupciones

```

Es decir, si nos fijamos en el contenido `SECTIONS` de nuestro *linker script*, veremos que lo primero que coloca en el binario es `.isr_vector`. A su vez, esta sección tiene, en primer lugar, el valor inicial de la pila, y la dirección de comienzo `Reset_Handler`. El procesador ya es conocedor de esto [pm0253 2.1.3](#) y cargará ambos al resetearse.

Se ofrece a modo ilustrativo esta visualización del espacio de memoria de nuestra aplicación, construido desde el linker script y el código de inicialización:

Address (Flash)	Contents	Description
0x08000000	<code>.isr_vector</code>	Vector Table: Initial SP, Reset_Handler, ISR Handlers
0x08000000	<code>_estack</code> (0x20040000)	Initial Stack Pointer Value (Top of RAM)
0x08000004	<code>Reset_Handler</code>	Initial Execution Entry Address (Reset Vector)
0x08000008	<code>NMI_Handler</code>	Non-Maskable Interrupt Handler
0x0800000C	<code>HardFault_Handler</code>	Hardware Fault Exception Handler
0x08000010+	Core Exceptions	MemManage, BusFault, UsageFault, SVC, SysTick
0x08000040+	Peripheral IRQs	WWDG, PVD, RTC, Flash, RCC, EXTI, DMA, TIM, etc.
0x08000...	<code>.text</code>	Executable Program Code
...	<code>.rodata</code>	Constant Data, String Literals, Read-Only Variables
...	<code>.data</code>	Initialized global variables
...	...	Various code utilities
Address (RAM)	Contents	Description
0x20000000	<code>.data</code>	Initialized Global/Static Data (Copied from Flash)
...	<code>.bss</code>	Uninitialized Global Data (Zero-filled by startup)
...	Dynamic Heap	Minimum Reserved Size: 0x200 (512 Bytes) ↑ Grows Up
...	Unallocated	Available RAM for Dynamic Stack/Heap Overlap
0x2003FC00	Main Stack	Minimum Reserved Size: 0x400 (1024 Bytes) ↓ Grows Down
0x20040000	<code>_estack</code>	Highest Stack Address (End of RAM)

Observamos que, como habíamos dicho, el primer valor que se encuentra en nuestro programa, en `0x08000000` es la dirección de la pila, para a continuación tener en `0x08000004` la dirección inicial a la que saltar (es decir, un puntero). Así pues, si en una dirección de memoria encontramos esta estructura, o similar, podemos asumir que en ese lugar tenemos un programa guardado. Nuestra función de comprobación de existencia de programa puede ser, por ejemplo, la siguiente:

```

1 //Check for app presence at a given location
2 uint8_t IsApplicationValid(uint32_t address) {
3     uint32_t sp = *(volatile uint32_t *)address;
4
5     // Check if initial Stack Pointer points into valid SRAM range (0x200XXXXX)
6     if ((sp & 0x2FF00000) == 0x20000000) {
7         return 1;
8     }
9     return 0;
10 }

```

Ahora, una vez detectada una aplicación en una ubicación concreta de memoria, debemos conseguir saltar a ella. Lo más importante en este punto es saber que tenemos que dejar el procesador en un estado totalmente neutro, ya que de lo contrario, la nueva aplicación podría sufrir comportamientos inesperados. Imaginemos por ejemplo que no desactivamos las interrupciones o ciertos periféricos: el procesador seguiría ejecutando funcionalidad totalmente ajena a nuestra aplicación.

Por tanto, lo primero es desactivar cualquier cosa que hayamos utilizado. En nuestro caso, el *ticker* del sistema y los periféricos `GPIO` y `UART`.

```

1 void JumpToApplication(uint32_t address) {

```

```

2 // Disable any active interrupts (only SysTick in our case)
3 SysTick_Disable();
4
5 // Disable the peripherals we were using
6 RCC_AHB1ENR->bits.GPIOAEN = 1;
7 RCC_AHB1ENR->bits.GPIOCEN = 1;
8 RCC_AHB1ENR->bits.GPIODEN = 1;
9 RCC_APB1ENR->bits.USART5EN = 1; // comm with espressif
10 RCC_APB2ENR->bits.USART6EN = 1; // comm with computer (stlink)

```

Por otro lado, tenemos que reubicar también el puntero de la pila para que se ubique al principio del nuevo rango requerido. De lo contrario, estaríamos trabajando con la pila en direcciones de memoria desconocidas, lo cual puede traer fallos de acceso a memoria o, aún peor, sobrescribir nuestro nuevo código si hubiera solape. El valor inicial del **SP** se extrae de la dirección inicial del programa:

```

1 // Set Main Stack Pointer (MSP)
2 __asm volatile ("msr msp, %0" : : "r" (*(volatile uint32_t *)address) : );

```

Ahora ya está todo listo para saltar a la función inicial (que recordemos es el **Reset_Handler** del archivo de inicialización **startup_stm32f723ieqx.s**). Como su dirección inicial se encuentra en la segunda palabra del binario (**address + 4**), la extraemos de ahí, y la convertimos a un tipo de función **void** antes de llamarla.

```

1 // Branch to reset handler of target firmware
2 typedef void (*pFunction)(void);
3 uint32_t JumpAddress = *(volatile uint32_t *)(address + 4);
4 pFunction JumpToApp = (pFunction)JumpAddress;
5 JumpToApp();

```

A partir de aquí, habremos perdido completamente el control del programa, y toda la ejecución queda a merced de la aplicación objetivo.

9.3.3. La memoria Flash

Nuestro código se almacenará en la memoria Flash del procesador. Esta memoria tiene la capacidad, al contrario de la RAM, de retener el contenido incluso tras apagarse y encenderse el procesador. Sin embargo, su uso por el programador está restringido (básicamente para evitar accidentalmente sobrescribir el código) así que tenemos que permitir los accesos en ella si queremos hacerlo. Echa un vistazo a la documentación de la **FLASH** en **rm0431 3.7.2** y completa el código de desbloqueo de la flash:

```

1 /* Flash Keys */
2 #define FLASH_KEY1          0x45670123U
3 #define FLASH_KEY2          0xCDEF89ABU
4 void Flash_Unlock(void) {
5     // ... completar si la flash está bloqueada (CR)
6     // ... desbloquear con la secuencia de claves en KEYR
7 }

```

Observa también la función **ESP01S_DownloadAndFlash** y verás que el flujo de programación es, a grandes rasgos, el siguiente:

```

1 Flash_Unlock();
2 Flash_ClearErrors();

```



```

3  for (bytes to write)
4      Flash_WriteByteRaw(current_flash_addr++, byte);
5  Flash_Lock();

```

Es decir, nos aseguramos de que la protección de la flash solo se quita en el fragmento de código que flashea el binario descargado, y en ningún otro sitio, para evitar errores.

9.3.4. Programación del *bootloader*

Ahora, con el archivo descargado y las funciones auxiliares de salto a programa, debemos programar el *bootloader*. Básicamente vamos a crear un código `main.c` que ceda el control a una región de memoria específica (en concreto, la `0x08010000` como vimos anteriormente). Completa el siguiente código para ello, añadiendo además los mensajes de depuración (Ver práctica 3) que consideres oportunos. Utiliza todas las funciones programadas anteriormente.

```

1  int main(void) {
2      // ... completar
3      // ... inicializar systick, periféricos y botón de usuario
4
5      uint8_t app_valid = IsApplicationValid(ADDR_FLASH_SECTOR_4);
6      uint8_t button_pressed = GPIO_PIN_READ(GPIOA, 0);
7      // Boot decision
8      // ... completar
9      // ... si el botón no está pulsado, comprobar si la app es válida y saltar a
10     ella
11     // ... si el botón está pulsado, comprobar de nuevo tras 2 segundos
12     // ... si sigue pulsado, en ese caso borrar la flash
13     // ... en cualquier otro caso, continuar con la descarga de la app
14
15     // Initialize ESP8266 device
16     if (!ESP01S_Init()) {
17         while (1);
18     }
19
20     // Connect Wi-Fi
21     if (!ESP01S_ConnectWiFi()) {
22         while (1);
23     }
24
25     // We must erase before flashing new stuff, since bits only
26     // change from 1 to 0 and not vice versa when flashing.
27     // This ensures initial state is 0xffffffff
28     Flash_EraseSector(FLASH_TARGET_SECTOR);
29
30     // Download over TCP and Flash directly
31     if (!ESP01S_DownloadAndFlash()) {
32         while (1);
33     }
34
35     JumpToApplication(ADDR_FLASH_SECTOR_4);
36 }

```

9.3.5. Compilación de la aplicación

Con esta práctica se proporciona un fichero binario compilado de ejemplo. No obstante, si queremos compilar el nuestro propio (con otro programa) tenemos que seguir unos pasos para asegurar que funcione.

Lo primero es ceder el control completamente de forma correcta. Hasta ahora, lo “único” que hemos hecho ha sido saltar a la dirección de la función inicial **Reset_Handler** de la aplicación objetivo. Pero, y todas las interrupciones? Si ahora activásemos una interrupción, saltaría a la tabla de interrupciones de la aplicación original, fallando por completo el código. Como vimos anteriormente, la aplicación tiene al comienzo una tabla de vectores que indican las direcciones de memoria de las diferentes interrupciones y código de arranque. El Bloque de Control del Sistema **SCB** (**pm0253 4.3**) guarda en su registro **VTOR** la dirección inicial de dicha tabla (es decir, la dirección inicial de nuestro código). Así pues, debemos cambiarla para reubicar todas las interrupciones de una sola tacada. Esto lo podemos hacer redefiniendo la función **System_Init** (definida como **.weak** precisamente para que nosotros la redefinamos y le demos funcionalidad si es necesario).

```
1 void SystemInit(void) {  
2     SCB->VTOR = 0x08010000;  
3 }
```

Por otro lado, tendremos que indicar en el linker script **STM32F723IEKX_FLASH.ld** el cambio de dirección inicial:

```
1 FLASH      (rx)      : ORIGIN = 0x8010000,   LENGTH = 256K
```

Finalmente, activamos la generación del archivo binario desde **C/C++ Build >Settings >MCU/MPU Post build outputs >Convert to binary file (-O binary)**.

Al compilar, se nos generará ahora un archivo **.bin** que es el que habrá que flashear.

9.3.6. Configuración del servidor de descarga

Llegados a este punto, deberíamos tener un bootloader funcional que... no puede hacer nada pues no tiene un servidor al que conectarse. Necesitamos hacer dos cosas:

1. Asegurarnos de que el módulo ESP-01S está conectado a nuestra placa.
2. Configurar un servidor para que se descargue el archivo binario.

Lo primero es relativamente sencillo, simplemente hay que colocar el módulo en el conector CN14 que se encuentra en la placa (conector con 8 pines huecos), en la dirección que indica la marca blanca inferior.

Respecto a lo segundo, se puede hacer de muchas maneras. La más cómoda es hacerlo desde nuestro propio teléfono, habilitando un punto inalámbrico que actúe como entrada a la red para nuestro programa. Asegúrate de configurar correctamente la conexión creando el archivo **ESP01S_config.h**

```

1 #define WIFI_SSID      "<nombre de tu red wifi>"
2 #define WIFI_PASSWORD  "<password de tu red wifi>"
3 #define SERVER_IP      "<ip de tu servidor>"
4 #define SERVER_PORT    <puerto de tu servidor>
5 #define FILE_PATH      "<nombre del fichero>"

```

Lo siguiente es abrir un servidor **http** en nuestro teléfono que sirva el binario **hello.bin** o cualquier otro que hayamos creado a mano.

- En Android, podemos hacerlo descargando la aplicación SHTTPS <https://play.google.com/store/apps/details?id=com.phlox.simpleserver&hl=es> y configurándola como vemos en la Figura 28. Únicamente tendremos que iniciar el servidor, poner la **Root Folder** apuntando a donde tengamos guardado el archivo binario de la aplicación, y seleccionar la interfaz de nuestra red inalámbrica (normalmente llamada **ap_XXX**) para que nos de la IP que tendremos que poner en la configuración anterior.
- En IOS, hay aplicaciones similares como <https://apps.apple.com/es/app/simple-server-http-server/id6443893597> o <https://apps.apple.com/es/app/share-server/id6754194116>.

Y listo! Ahora nuestra aplicación debería ser capaz de conectarse al punto WiFi de nuestro teléfono, descargar el binario, flashearlo, y lanzarlo a ejecución! Asegúrate de poner mensajes de depuración si tienes problemas, para ver dónde pueda haber dificultades.

9.4. Para hacer más

En ocasiones, en lugar de tener una aplicación *bootloader* que se encarga de descargar el binario actualizado, se fusionan ambas funcionalidades en una sola (*bootloader* + app). Así, el propio *bootloader* también es actualizado. Para ello, se reservan dos regiones de flash diferentes, *A* y *B*. La aplicación *A*, cuando detecte una actualización (a través de WiFi por ejemplo), la escribirá en *B*, y configurará el procesador para arrancar desde ahí la próxima vez. Y viceversa, si la *B* es la que está en juego, actualizará *A*. Así, incluso si durante el proceso de actualización fallara algo, siempre habrá una de las dos aplicaciones funcional. Investiga cómo se haría este proceso en nuestro procesador **STM32F723IE**.

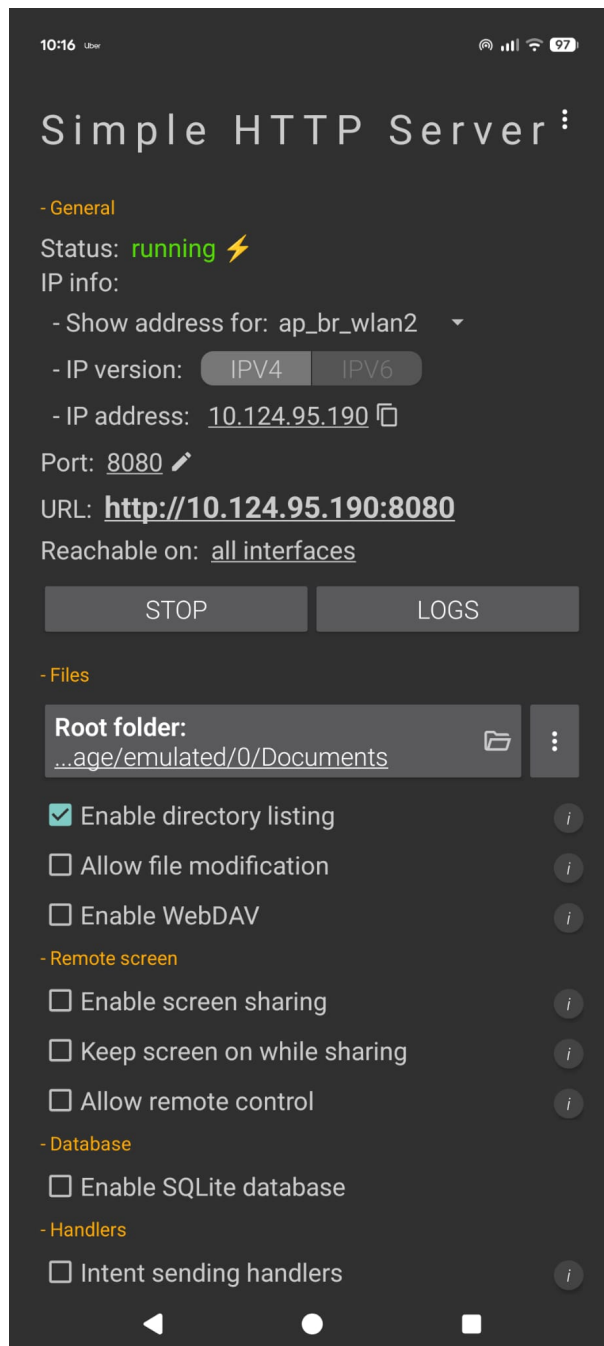


Figura 28: Configuración SHTTPS Android